

Project Management

By Erik W. Larson and Clifford F. Gray

Chapter 4: “Defining the Project”

Finally, the matrix is useful midway in the project for approaching a problem that must be solved.

One caveat must be mentioned; during the course of a project, priorities may change. The customer may suddenly need the project completed one month sooner, or new directives from top management may emphasize cost saving initiatives. The project manager needs to be vigilant in order to anticipate and confirm changes in priorities and make appropriate adjustments.

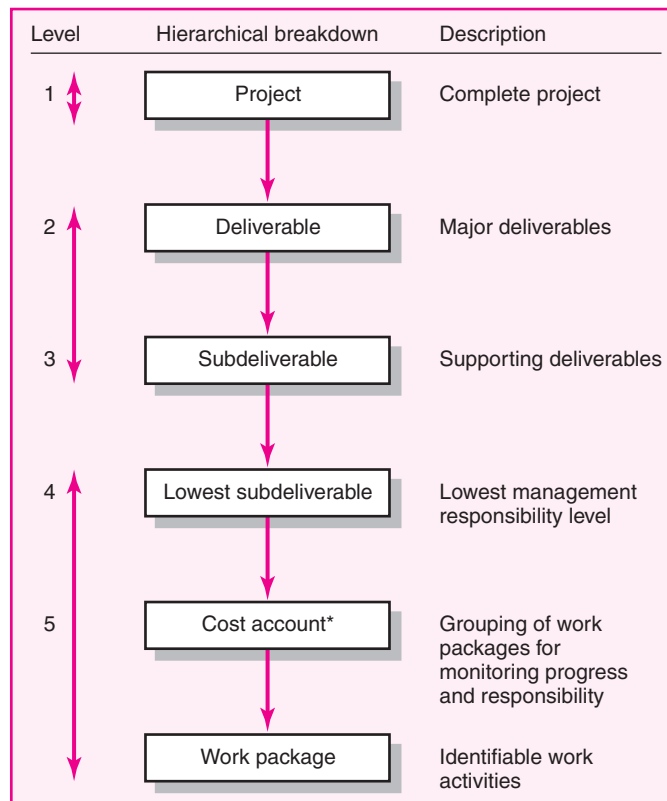
Step 3: Creating the Work Breakdown Structure

Major Groupings Found in a WBS

Once the scope and deliverables have been identified, the work of the project can be successively subdivided into smaller and smaller work elements. The outcome of this hierarchical process is called the **work breakdown structure (WBS)**. The WBS is a map of the project. Use of WBS helps to assure project managers that all products and work elements are identified, to integrate the project with the current organization, and to establish a basis for control. Basically, the WBS is an outline of the project with different levels of detail.

Figure 4.3 shows the major groupings commonly used in the field to develop a hierarchical WBS. The WBS begins with the project as the final deliverable.

FIGURE 4.3
Hierarchical
Breakdown
of the WBS



* This breakdown groups work packages by type of work within a deliverable and allows assignment of responsibility to an organizational unit. This extra step facilitates a system for monitoring project progress (discussed in Chapter 13).

Major project work deliverables/systems are identified first; then the subdeliverables necessary to accomplish the larger deliverables are defined. The process is repeated until the subdeliverable detail is small enough to be manageable and where one person can be responsible. This subdeliverable is further divided into work packages. Because the lowest subdeliverable usually includes several work packages, the work packages are grouped by type of work—for example, hardware, programming, testing. These groupings within a subdeliverable are called cost accounts. This grouping facilitates a system for monitoring project progress by work, cost, and responsibility.

How WBS Helps the Project Manager

The WBS defines all the elements of the project in a hierarchical framework and establishes their relationships to the project end item(s). Think of the project as a large work package that is successively broken down into smaller work packages; the total project is the summation of all the smaller work packages. This hierarchical structure facilitates evaluation of cost, time, and technical performance at all levels in the organization over the life of the project. The WBS also provides management with information appropriate to each level. For example, top management deals primarily with major deliverables, while first-line supervisors deal with smaller subdeliverables and work packages.

Each item in the WBS needs a time and cost estimate. With this information it is possible to plan, schedule, and budget your project. The WBS also serves as a framework for tracking cost and work performance.

As the WBS is developed, organizational units and individuals are assigned responsibility for executing work packages. This integrates the work and the organization. In practice, this process is sometimes called the organization breakdown structure (OBS), which will be further discussed later in the chapter.

Use of the WBS provides the opportunity to “roll up” (sum) the budget and actual costs of the smaller work packages into larger work elements so that performance can be measured by organizational units and work accomplishment.

The WBS can also be used to define communication channels and assist in understanding and coordinating many parts of the project. The structure shows the work and organizational units responsible and suggests where written communication should be directed. Problems can be quickly addressed and coordinated because the structure integrates work and responsibility.

WBS Development

Figure 4.4 on page 112 shows a simplified WBS for development of a new personal computer project. At the top of the chart (level 1) is the project end item—a deliverable product or service. Note how the levels of the structure can represent information for different levels of management. For example, level 1 information represents the total project objective and is useful to top management; levels 2, 3, and 4 are suitable for middle management; and level 5 is for first-line managers.

Level 2 shows a partial list of deliverables necessary to develop the personal computer. One deliverable is the disk storage unit (shaded), which is made up of three subdeliverables—external USB, optical, and hard disks. Finally, the hard disk requires four subdeliverables—motor, circuit board, chassis frame, and read/write head. These subdeliverables represent the lowest manageable elements

SNAPSHOT FROM PRACTICE

London 2012 Olympic Games



On July 27, 2012, the London Olympic Games will start. Olympic Delivery Authority (ODA), the Olympics' project client, makes it clear: "Not starting on that day is not an option. The

deadline is 100 percent fixed!" This is the mandate for both the general contractors and the information services (IS) teams. Each must meet the schedule while contributing to the overall objectives of the Olympic team.



© PA Photos/Landov

2012 Olympic Project Team Objectives:

- To stage the 2012 Olympic and Paralympic games: 27 July–12 August, 2012;
- To deliver an Olympic park and related venues;
- To maximize economic, social, health, and environmental benefits for London and for the UK;
- To provide sustained improvement in UK sports pre and post games;

of the project. Each subdeliverable requires work packages that will be completed by an assigned organizational unit. Each deliverable will be successively divided in this manner. It is not necessary to divide all elements of the WBS to the same level.

The lowest level of the WBS is called a **work package**. Work packages are short-duration tasks that have a definite start and stop point, consume resources, and represent cost. Each work package is a control point. A work package manager is

- To ensure the venues are used for something useful post games.

Below is the initial timeline for the 2012 Olympic event:

2006: Set foundations
 2007: Strategic planning
 2008: Review all past Olympics
 2010: Operational planning
 2011: Test events
 2012: Operational readiness
 2013: Close down

Source: ComputerWeekly.com

Physical facilities and information services are two of the major areas that need massive coordination and represent a sizable part of the total budget.

London 2012 will be the sixth Olympic Games that Atos Origin will design, build and operate the IS infrastructure. Michele Hyron, their chief integrator responsible for the event's information systems, will be overseeing one of the biggest information technology projects ever. She is a seasoned Olympics project manager who has served as IS operations manager in previous Olympic Games in Athens and Beijing. By examining the processes and lessons learned in the earlier games, she hopes past mistakes can be avoided. For example, from the Beijing games four lessons learned were better training of support staff, freezing design four months before games, isolating IT from the Internet, and better planning.

Hyron carries over about 40 to 50 percent of systems planning from one Olympics to the next, and then adjusts to local conditions. Hyron has designed and is building fully redundant systems for 2012 that will be used for two years of testing of over 1,000 scenarios to study how the systems and technical personnel respond. Test scenarios might include a security breach, a fire in a living facility, staff contracting food poisoning, and events delayed.

Physical facilities and logistics are equally as important as IS infrastructure. The Olympic Delivery Authority (ODA) selected EDAW Consortium to design a master plan for the Olympic Park's infrastructure, including utilities, waterways, landscape, platforms for the site, roads, and bridges.

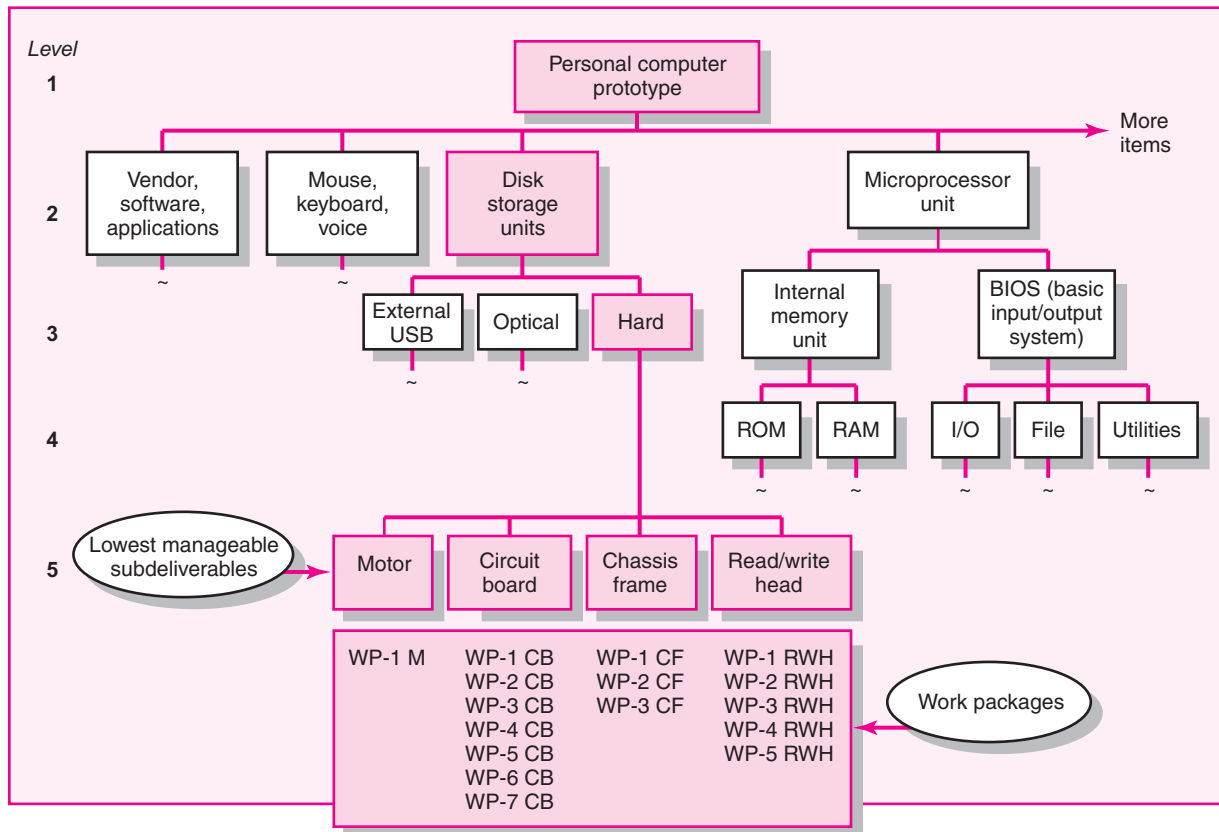
A 500-acre Olympic Park in Stratford, east London, will be the epicenter of the Games. The Olympic Stadium will house the 80,000-seat coliseum as well as the aquatics center that has two pools and a diving pool. A Channel Tunnel Rail Link (CTRL) will carry a high-speed shuttle service between central London and the Olympic Park in just seven minutes. This link will also connect with service to continental Europe. Transportation to the Olympics will be supported with improved underground services. Planners expect to have a train arriving at the Olympic Park every 15 seconds. There will be 20 km of new roads and more than 30 new bridges to connect the Olympic Park with nearby communities. Prime Minister Gordon Brown estimates that nearly 30,000 workers will build the Olympic Park and Olympic Village.

From the outset, cost estimating has been a challenge. When London announced its bid for the 2012 Games, the estimated cost for the games was £4 billion. By 2007 the estimated costs climbed to £9.325 billion. In mid-2009, 500 industry professionals estimate the costs will rise to £11.6 billion. Some estimates have run as high as £12.6 billion.

The National Audit Office study identified two major problems with the Olympic Games project: no one single individual is in charge and there is no proper budget. In addition, the committee offered the following suggestions: (a) clarify key deliverables and expected costs, (b) establish a baseline for budget control, and (c) manage both contingency and project funds more rigorously.

The Olympics Games are often called "the greatest show on earth." For the project managers of the 2012 Olympics, there are many challenges and opportunities to be addressed before the Games are over.

responsible for seeing that the package is completed on time, within budget, and according to technical specifications. Practice suggests a work package should not exceed 10 workdays or one reporting period. If a work package has a duration exceeding 10 days, check or monitoring points should be established within the duration, say, every three to five days, so progress and problems can be identified before too much time has passed. Each work package of the WBS should be as independent of other packages of the project as possible. No work package is described in more than one subdeliverable of the WBS.

FIGURE 4.4 Work Breakdown Structure

There is an important difference from start to finish between the last work breakdown subdeliverable and a work package. Typically, a work breakdown subdeliverable includes the outcomes of more than one work package from perhaps two or three departments. Therefore, the subdeliverable does not have a duration of its own and does not consume resources or cost money directly. (In a sense, of course, a duration for a particular work breakdown element can be derived from identifying which work package must start first [earliest] and which package will be the latest to finish; the difference from start to finish becomes the duration for the subdeliverable.) The higher elements are used to identify deliverables at different phases in the project and to develop status reports during the execution stage of the project life cycle. Thus, the work package is the basic unit used for planning, scheduling, and controlling the project.

To review, each work package in the WBS

1. Defines work (what).
2. Identifies time to complete a work package (how long).
3. Identifies a time-phased budget to complete a work package (cost).
4. Identifies resources needed to complete a work package (how much).

5. Identifies a single person responsible for units of work (who).
6. Identifies monitoring points for measuring progress (how well).

Creating a WBS from scratch can be a daunting task. Project managers should take advantage of relevant examples from previous projects to begin the process.

WBSs are products of group efforts. If the project is small, the entire project team may be involved breaking down the project into its components. For large, complex projects, the people responsible for the major deliverables are likely to meet to establish the first two levels of deliverables. In turn, further detail would be delegated to the people responsible for the specific work. Collectively this information would be gathered and integrated into a formal WBS by a project support person. The final version would be reviewed by the inner echelon of the project team. Relevant stakeholders (most notably customers) would be consulted to confirm agreement and revise when appropriate.

Project teams developing their first WBS frequently forget that the structure should be end-item, output oriented. First attempts often result in a WBS that follows the organization structure—design, marketing, production, finance. If a WBS follows the organization structure, the focus will be on the organization function and processes rather than the project output or deliverables. In addition, a WBS with a process focus will become an accounting tool that records costs by function rather than a tool for “output” management. Every effort should be made to develop a WBS that is output oriented in order to concentrate on concrete deliverables. See Snapshot from Practice: Creating a WBS for more advice on creating a WBS. This process is discussed next.

Step 4: Integrating the WBS with the Organization

The WBS is used to link the organizational units responsible for performing the work. In practice, the outcome of this process is the **organization breakdown structure (OBS)**. The OBS depicts how the firm has organized to discharge work responsibility. The purposes of the OBS are to provide a framework to summarize organization unit work performance, identify organization units responsible for work packages, and tie the organizational unit to cost control accounts. Recall, cost accounts group similar work packages (usually under the purview of a department). The OBS defines the organization subdeliverables in a hierarchical pattern in successively smaller and smaller units. Frequently, the traditional organization structure can be used. Even if the project is completely performed by a team, it is necessary to break down the team structure for assigning responsibility for budgets, time, and technical performance.

As in the WBS, the OBS assigns the lowest organizational unit the responsibility for work packages within a cost account. Herein lies one major strength of using WBS and OBS; they can be *integrated* as shown in Figure 4.5. The intersection of work packages and the organizational unit creates a project control point (**cost account**) that integrates work and responsibility. The intersection of the WBS and OBS represents the set of work packages necessary to complete the subdeliverable located immediately above and the organizational unit on the left responsible for accomplishing the packages at the intersection. Later we will use the intersection as a cost account for management control of projects. For example, the circuit

Agile Project Management Using Scrum

By Ken Schwaber

Chapter 1: “Backdrop: The Science
of Scrum”

The Skeleton and Heart of Scrum

Scrum hangs all of its practices on an iterative, incremental process skeleton. Scrum's skeleton is shown in [Figure 1-2](#). The lower circle represents an iteration of development activities that occur one after another. The output of each iteration is an increment of product. The upper circle represents the daily inspection that occurs during the iteration, in which the individual team members meet to inspect each others' activities and make appropriate adaptations. Driving the iteration is a list of requirements. This cycle repeats until the project is no longer funded.

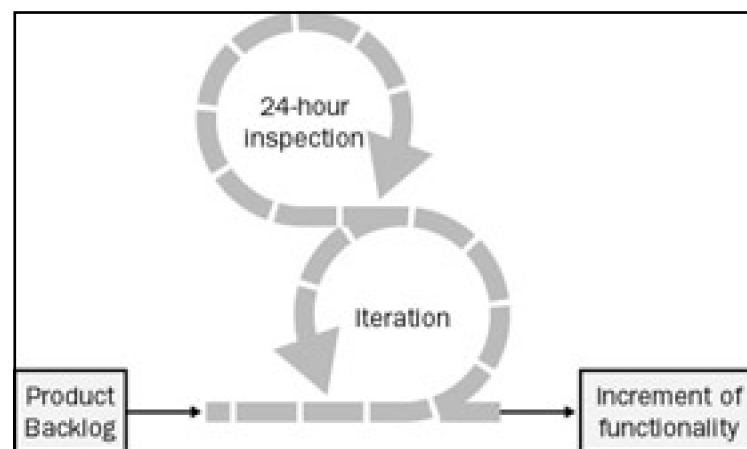


Figure 1-2: Scrum skeleton

The skeleton operates this way: At the start of an iteration, the team reviews what it must do. It then selects what it believes it can turn into an increment of potentially shippable functionality by the end of the iteration. The team is then left alone to make its best effort for the rest of the iteration. At the end of the iteration, the team presents the increment of functionality it built so that the stakeholders can inspect the functionality and timely adaptations to the project can be made.

The heart of Scrum lies in the iteration. The team takes a look at the requirements, considers the available technology, and evaluates its own skills and capabilities. It then collectively determines how to build the functionality, modifying its approach daily as it encounters new complexities, difficulties, and surprises. The team figures out what needs to be done and selects the best way to do it. This creative process is the heart of the Scrum's productivity.

Scrum implements this iterative, incremental skeleton through three roles. I'll provide a quick overview of these people operating within the Scrum process. Then I'll describe the Scrum process flow and its artifacts. [Appendix A, ?Rules,?](#) and [Appendix B, ?Definitions,?](#) provide a list of rules as well as Scrum definitions that can be referred to as you read this book. More detailed information about Scrum can be found in [Appendix C, ?Resources,?](#) and in Ken Schwaber and Mike Beedle's, *Agile Software Development with Scrum* (Prentice Hall, 2002).

Scrum Roles

There are only three Scrum roles: the Product Owner, the Team, and the ScrumMaster. All management responsibilities in a project are divided among these three roles. The Product Owner is responsible for representing the interests of everyone with a stake in the project and its resulting system. The Product Owner achieves initial and ongoing funding for the project by creating the project's initial overall requirements, return on investment (ROI) objectives, and release plans. The list of requirements is called the Product Backlog. The Product Owner is responsible for using the Product Backlog to ensure that the most valuable functionality is produced first and built upon; this is achieved by frequently prioritizing the Product Backlog to queue up the most valuable requirements for the next iteration. The Team is responsible for developing functionality. Teams are self-managing, self-organizing, and cross-functional, and they are responsible for figuring out how to turn Product Backlog into an increment of functionality within an iteration and managing their own work to do so. Team members are collectively responsible for the success of each iteration and of the project as a whole. The ScrumMaster is responsible for the Scrum process, for teaching Scrum to everyone involved in the project, for implementing Scrum so that it fits within an organization's culture and still delivers the expected benefits, and for ensuring that everyone follows Scrum rules and practices.

The people who fill these roles are those who have committed to the project. Others might be interested in the project, but they aren't on the hook. Scrum makes a clear distinction between these two groups and ensures that those who are responsible for the project have the authority to do what is necessary for its success and that those who aren't responsible can't interfere unnecessarily. Throughout this book, I refer to these people as "pigs" and "chickens," respectively. These names come from an old joke: A chicken and a pig are walking down the road. The chicken says to the pig, "Do you want to open a restaurant with me?" The pig considers the question and replies, "Yes, I'd like that. What do you want to call the restaurant?" The chicken replies, "Ham and Eggs!" The pig stops, pauses, and replies, "On second thought, I don't think I want to open a restaurant with you. I'd be committed, but you'd only be involved."

This distinction is important in Scrum and is relevant to Scrum's insistence upon total visibility. It should always be clear who is on the hook and who is just a kibitzer. Who is responsible for the ROI, and who has a stake in the ROI but isn't accountable? Who has to turn difficult technology into functionality, and who is a troublesome "devil's advocate"? The rules of Scrum distinguish between the chickens and the pigs to increase productivity, create momentum, and put an end to floundering.

Scrum Flow

A Scrum project starts with a vision of the system to be developed. The vision might be vague at first, perhaps stated in market terms rather than system terms, but it will become clearer as the project moves forward. The Product Owner is responsible to those funding the project for delivering the vision in a manner that maximizes their ROI. The Product Owner formulates a plan for doing so that includes a Product Backlog. The Product Backlog is a list of functional and nonfunctional requirements that, when turned into functionality, will deliver this vision. The Product Backlog is prioritized so that the items most likely to generate value are top priority and is divided into proposed releases. The prioritized Product Backlog is a starting point, and the contents, priorities, and grouping of the Product Backlog into releases usually changes the moment the project starts—as should be expected. Changes in the Product Backlog reflect changing business requirements and how quickly or slowly the Team can transform Product Backlog into functionality.

All work is done in Sprints. Each Sprint is an iteration of 30 consecutive calendar days. Each Sprint is initiated with a Sprint planning meeting, where the Product Owner and Team get together to collaborate about what will be done for the next Sprint. Selecting from the highest priority Product Backlog, the Product Owner tells the Team what is desired, and the Team tells the Product Owner how much of what is desired it believes it can turn into functionality over the next Sprint. Sprint planning meetings cannot last longer than eight hours—that is, they are time-boxed to avoid too much hand-wringing about what is possible. The goal is to get to work, not to think about working.

The Sprint planning meeting has two parts. The first four hours are spent with the Product Owner presenting the highest priority Product Backlog to the Team. The Team questions him or her about the content, purpose, meaning, and intentions of the Product Backlog. When the Team knows enough, but before the first four hours elapses, the Team selects as much Product Backlog as it believes it can turn into a completed increment of potentially shippable product functionality by the end of the Sprint. The Team commits to the Product Owner that it will do its best. During the second four hours of the Sprint planning meeting, the Team plans out the Sprint. Because the Team is responsible for managing its own work, it needs a tentative plan to start the Sprint. The tasks that compose this plan are placed in a Sprint Backlog; the tasks in the Sprint Backlog emerge as the Sprint evolves. At the start of the second four-hour period of the Sprint planning meeting, the Sprint has started, and the clock is ticking toward the 30-day Sprint time-box.

Every day, the team gets together for a 15-minute meeting called a Daily Scrum. At the Daily Scrum, each Team member answers three questions: What have you done on this project since the last Daily Scrum meeting? What do you plan on doing on this project between now and the next Daily Scrum meeting? What impediments stand in the way of you meeting your commitments to this Sprint and this project? The purpose of the meeting is to synchronize the work of all Team members daily and to schedule any meetings that the Team needs to forward its progress.

At the end of the Sprint, a Sprint review meeting is held. This is a four-hour, time-boxed meeting at which the Team presents what was developed during the Sprint to the Product Owner and any other stakeholders who want to attend. This informal meeting at which the functionality is presented is intended to bring people together and help them collaboratively determine what the Team should do next. After the Sprint review and prior to the next Sprint planning meeting, the ScrumMaster holds a Sprint retrospective meeting with the Team. At this three-hour, time-boxed meeting, the ScrumMaster encourages the Team to revise, within the Scrum process framework and practices, its development process to make it more effective and enjoyable for the next Sprint. Together, the Sprint planning meeting, the Daily Scrum, the Sprint review, and the Sprint retrospective constitute the empirical inspection and adaptation practices of Scrum. Take a look at [Figure 1-3](#) to see a diagram of the Scrum process.

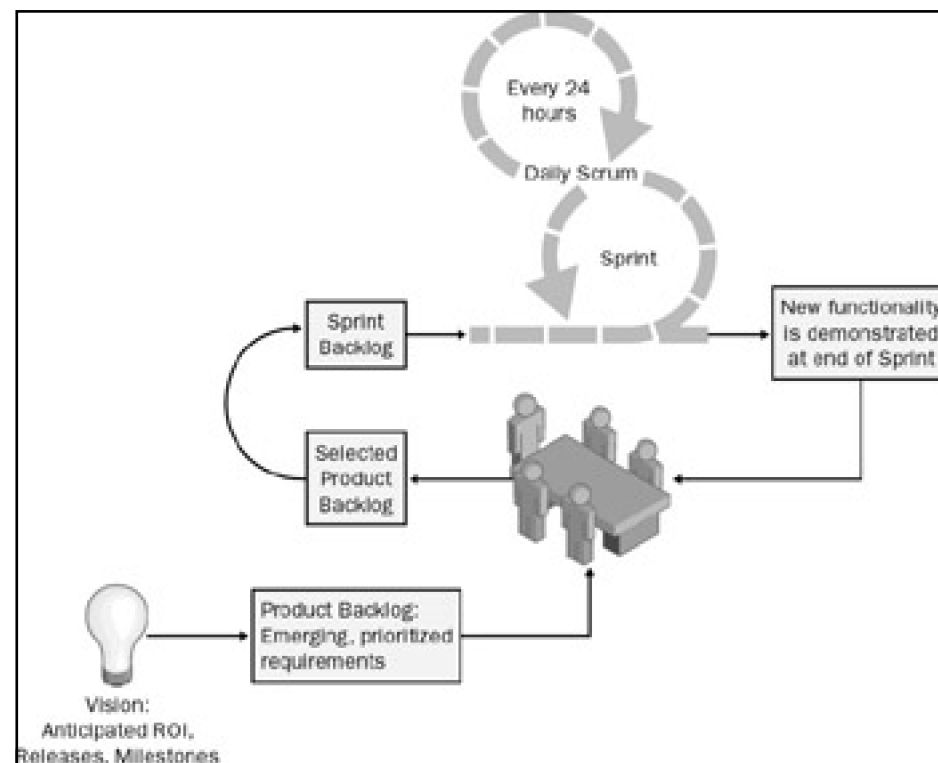


Figure 1-3: Scrum process overview

Scrum Artifacts

Scrum introduces a few new artifacts. These are used throughout the Scrum process and are described in the following sections.

Product Backlog

The requirements for the system or product being developed by the project(s) are listed in the Product Backlog. The Product Owner is responsible for the contents, prioritization, and availability of the Product Backlog. The Product Backlog is never complete, and the Product Backlog used in the project plan is merely an initial estimate of the requirements. The Product Backlog evolves as the product and the environment in which it will be used evolves. The Product Backlog is dynamic; management constantly changes it to identify what the product needs to be appropriate, competitive, and useful. As long as a product exists, the Product Backlog also exists. An example of Product Backlog maintained on the Scrum Product Management tool, based in a spreadsheet, is shown in [Figure 1-4](#).

Backlog Description	Initial Estimate	Adjustment Factor	Adjusted Estimate	work remaining until completion						
				1	2	3	4	5	6	7
Title Import				256	289	133	140	140	140	140
Project selection or new	3	0.2	3.6	3.6	0	0	0	0	0	0
Template backlog for new projects	2	0.2	2.4	2.4	0	0	0	0	0	0
Create product backlog worksheet with formatting	3	0.2	3.6	3.6	0	0	0	0	0	0
Create sprint backlog worksheet with formatting	3	0.2	3.6	3.6	0	0	0	0	0	0
Display tree view of product backlog, releases, sprints	2	0.2	2.4	2.4	0	0	0	0	0	0
Sprint-1	13	0.2	15.6	16	0	0	0	0	0	0
Create a new window containing product backlog template	3	0.2	3.6	3.6	3.6	0	0	0	0	0
Create a new window containing sprint backlog template	2	0.2	2.4	2.4	2.4	0	0	0	0	0
Burndown window of product backlog	5	0.2	6	6	6	0	0	0	0	0
Burndown window of sprint backlog	1	0.2	1.2	1.2	1.2	0	0	0	0	0
Display tree view of product backlog, releases, prints	2	0.2	2.4	2.4	2.4	0	0	0	0	0
Display burndown for selected sprint or release	3	0.2	3.6	3.6	3.6	0	0	0	0	0
Sprint-2	16	0.2	19.2	19	19	1.2	0	0	0	0
Automatic recalculation of values and totals	3	0.2	3.6	3.6	3.6	3.6	0	0	0	0
As changes are made to backlog in secondary window, update burndown graph on main page	2	0.2	2.4	2.4	2.4	2.4	0	0	0	0
Hide/automatic redisplay of burndown window	3	0.2	3.6	3.6	3.6	3.6	0	0	0	0
Insert Sprint capability ... adds summary Sprint row	2	0.2	2.4	2.4	2.4	2.4	0	0	0	0
Insert Release capability ... adds summary row for backlog in Sprint	1	0.2	1.2	1.2	1.2	1.2	0	0	0	0
Owner/assigned capability and column's optional	2	0.2	2.4	2.4	2.4	2.4	0	0	0	0
Print burndown graphs	1	0.2	1.2	1.2	1.2	1.2	0	0	0	0
Sprint-3	14	0.2	16.8	17	17	17	0	0	0	0
Duplicate incomplete backlog without affecting totals	5	0.2	6	6	6	6	6	6	6	6
Note capability	6	0.2	7.2	7.2	7.2	7.2	7.2	7.2	7.2	7.2
What-if release capability on burndown graph	15	0.2	18	18	18	18	18	18	18	18
Trend capability on burndown server	3	0.2	3.6	3.6	3.6	3.6	3.6	3.6	3.6	3.6
Publish facility for entire project, publishing it as HTML web pages	11	0.2	13.2	0	0	13	13	13	13	13
Future Sprints	39	0.2	46.8	34	34	47	47	47	47	47
Release-1				85	78	65	47	47	47	47

Figure 1-4: Product Backlog

This spreadsheet is the March 2003 Product Backlog for a project for developing the Scrum Project Management software. I was the Product Owner. The rows are the Product Backlog items, separated by Sprint and Release subheadings. For instance, all of the rows above Sprint 1 represent tasks that were worked on in that Sprint. The rows between the Sprint 1 and Sprint 2 subheadings were done in Sprint 2. Notice that the row Display Tree View Of Product Backlog, Releases, Sprints is duplicated in Sprint 1 and Sprint 2. This is because row 10 wasn't completed in Sprint 1, so it was moved down to the Sprint 2 for completion. If I decided that it was lower priority after Sprint 1, I could have moved it even lower in the priority list.

The first four columns are the Product Backlog item name, the initial estimate, the complexity factor, and the adjusted estimate. The complexity factor increases the estimate due to project characteristics that reduce the productivity of the Team. The remaining columns represent the Sprints during which the Product Backlog is developed. When the Product Backlog is first thought of and entered, its estimated work is placed into the column of the Sprint that is going on at that time. The developers and I devised most of the backlog items shown before starting this project. The sole exception is row 31 (Publish Facility For Entire Project, Publishing It As HTML Web Pages), which I didn't think of until Sprint 3.

A *burndown chart* shows the amount of work remaining across time. The burndown chart is an excellent way of visualizing the correlation between the amount of work remaining at any point in time and the progress of the project Team(s) in reducing this work. The intersection of a trend line for work remaining and the horizontal axis indicates the most probable

completion of work at that point in time. A burndown chart reflecting this is shown in [Figure 1-5](#). This allows me to “what if” the project by adding and removing functionality from the release to get a more acceptable date or extend the date to include more functionality. The burndown chart is the collision of reality (work done and how fast it’s being done) with what is planned, or hoped for.

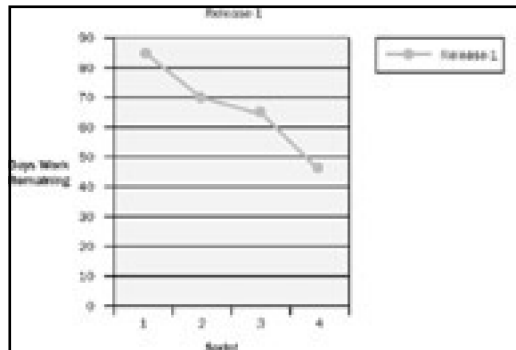


Figure 1-5: Burndown chart

The items in the Product Backlog for future Sprints are quite coarse- grained. I haven’t had the Team start work on these items, so I haven’t expended the time to analyze and more finely estimate them. Similarly, there are plenty more requirements for this product. They just haven’t been thought through. When I have the time or inclination to start development again, I’ll define more Product Backlog items. This is an example of the requirements for the product emerging. I can defer building an inventory of Product Backlog until I am ready to engage a Team to convert it to functionality.

Sprint Backlog

The Sprint Backlog defines the work, or tasks, that a Team defines for turning the Product Backlog it selected for that Sprint into an increment of potentially shippable product functionality. The Team compiles an initial list of these tasks in the second part of the Sprint planning meeting. Tasks should be divided so that each takes roughly 4 to 16 hours to finish. Tasks longer than 4 to 16 hours are considered mere placeholders for tasks that haven’t yet been appropriately defined. Only the Team can change the Sprint Backlog. The Sprint Backlog is a highly visible, real-time picture of the work that the Team plans to accomplish during the Sprint. An example Sprint Backlog is shown in [Figure 1-6](#). The rows represent Sprint Backlog tasks; the columns represent the 30 days in the Sprint. Once a task is defined, the estimated number of hours remaining to complete the task is placed in the intersection of the task and the Sprint day by the person working on the task.

Task Description	Originator	Responsible	Status (Not Started/In Progress/Completed)	Hours of work remaining until											
				1	2	3	4	5	6	7	8	9	10	11	12
Meet to discuss the goals and features for Sprint 3-6	Danville	Danville/See	Completed	20	0	0	0	0	0	0	0	0	0	0	0
Move Calculations out of Crystal Reports	Jim	Allen	Not Started	8	8	8	8	8	8	8	8	8	8	8	8
Get REG Data		Tom	Completed	12	0	0	0	0	0	0	0	0	0	0	0
Analyze REG Data - Title		George	In Progress	24	24	24	24	12	16	10	10	10	10	10	10
Analyze REG Data - Parcel		Tim	Completed	12	12	12	12	12	4	4	4	0	0	0	0
Analyze REG Data - Encumbrance		Josh	In Progress							12	10	10	10	10	10
Analyze REG Data - Contact		Danville	In Progress	24	24	24	24	12	16	8	8	8	8	8	8
Analyze REG Data - Facilities		Allen	In Progress	24	24	24	24	12	16	10	10	10	10	10	10
Define & build Database		Barry/Dave	In Progress	80	80	80	80	80	80	40	40	60	60	60	60
Validate the size of the REG database		Tim	Not Started												
Look at REG Data on the G1		Dave	In Progress	3	3	3	3	3	3	3	3	3	3	3	3
Confirm agreement with REG		Sam	Not Started												
Confirm REG Staff Availability		Tom	Not Started	1	1	1	1	1	1	1	1	1	1	1	1
Switch JDL to 1.3.1.1. Run all tests		Allen	Not Started	8	8	8	8	8	8	8	8	8	8	8	8
Store PDF files in a structure		Jacquin	Completed	8	0	0	0	0	0	0	0	0	0	0	0
Toplink. Cannot get rid of netpage parser		Richard	Completed	4	0	0	0	0	0	0	0	0	0	0	0
Build test data repository		Barry	In Progress	10	18	10	10	10	16	10	10	8	8	8	8
Move application and database to Qual (incl Crystal)		Richard	Completed	4	4	4	4	4	4	4	0	0	0	0	0
Set up Crystal environment		Josh	Completed	2	2	2	2	1	1	1	6	0	0	0	0
Test App in Qual		See	In Progress												20
Defining sprint goal required for solution in 2002		Lynde	In Progress	40	40	40	40	40	40	40	38	38	38	38	38
Reference tables for import process		Josh	In Progress												
Build standard import scenarios process		Josh	In Progress									12	12	12	10
Handle multiple file imports on same page		Jacquin	Disregarded												
Migrate CruiseControl Servlet to R/S 6.0 (Jandoc_7101) server		Allen	Not Started	4	4	4	4	4	4	4	4	4	4	4	4
Create web server for Qual on RF108		Allen	Completed	1	0	0	0	0	0	0	0	0	0	0	0
LTC's Disk		Danville/ George	In Progress	12	12	12	12	8	8	8	8	8	8	8	8
Follow thru with questions about REG data to Sam/Tom, etc. Reg, LTD	Jacquin	Danville	Completed	10	18	10	10	10	8	8	0	0	0	0	0
Map REG data to Active Tables-- see also #34	Jacquin	Jacquin/Allen	In Progress	50	58	50	50	50	54	50	50	50	54	50	50
Preparer SQL to import from REG tables to Active Tables	Jacquin	George	In Progress	26	26	26	26	26	26	26	26	26	26	26	26

Figure 1-6: Sprint Backlog

Increment of Potentially Shippable Product Functionality

Scrum requires Teams to build an increment of product functionality every Sprint. This increment must be potentially shippable, because the Product Owner might choose to immediately implement the functionality. This requires that the increment consist of thoroughly tested, well-structured, and well-written code that has been built into an executable and that the user operation of the functionality is documented, either in Help files or in user documentation. This is the definition of a “done” increment.

If the product increment that is created during the Sprint has a more exacting use, the development organization usually defines the additional product requirements as standards or

conventions. For example, the Food and Drug Administration (FDA) approves all products that will be used in life-critical circumstances in healthcare settings. As part of the approval process, the FDA checks that the product requirements are adequate and complete and that the requirements can be directly traced to the product. For each increment of FDA life-critical products to be potentially shippable, these additional facets of the product must also be developed—so that each increment of the product is potentially ready for FDA approval.

Team LiB

◀ PREVIOUS

NEXT ▶

Team LiB

◀ PREVIOUS

NEXT ▶

Summary

There you have it: the theory, the skeleton, the heart, the roles, the flow, and the artifacts of Scrum. For more of the how, why, and what of Scrum, refer to the Appendixes for more resources, a glossary of terms, and the Scrum rules. But remember that just as my bike trips in eastern Massachusetts don't qualify me for the Tour de France, the knowledge I've imparted in this chapter doesn't qualify you to manage a project with Scrum. For this, you need some practice and some understanding of Scrum being applied in real situations. That's what the next chapters are about—the practice of Scrum.

Team LiB

◀ PREVIOUS

NEXT ▶

Agile Project Management Using Scrum

By Ken Schwaber

Appendix A: Rules

Appendix A: Rules

The ScrumMaster is responsible for ensuring that everyone related to a project, whether chickens or pigs, follows the rules of Scrum. These rules hold the Scrum process together so that everyone knows how to play. If the rules aren't enforced, people waste time figuring out what to do. If the rules are disputed, time is lost while everyone waits for a resolution. These rules have worked in literally thousands of successful projects. If someone wants to change the rules, use the Sprint retrospective meeting as a forum for discussion. Rule changes should originate from the Team, not management. Rule changes should be entertained if and only if the ScrumMaster is convinced that the Team and everyone involved understands how Scrum works in enough depth that they will be skillful and mindful in changing the rules. No rules can be changed until the ScrumMaster has determined that this state has been reached.

Sprint Planning Meeting

The Sprint planning meeting is time-boxed to 8 hours and consists of two segments that are time-boxed to 4 hours each. The first segment is for selecting Product Backlog; the second segment is for preparing a Sprint Backlog.

- The attendees are the ScrumMaster, the Product Owner, and the Team. Additional parties can be invited by any of these people to provide additional business domain or technology domain information and advice, but they are dismissed after this information is provided. There are no chickens as observers.
- The Product Owner must prepare the Product Backlog prior to the meeting. In the absence of either the Product Owner or the Product Backlog, the ScrumMaster is required to construct an adequate Product Backlog prior to the meeting and to stand in for the Product Owner.
- The goal of the first segment, or first 4 hours, is for the Team to select those Product Backlog items that it believes it can commit to turning into an increment of potentially shippable product functionality. The Team will demonstrate this functionality to the Product Owner and stakeholders at the Sprint review meeting at the end of the Sprint.
- The Team can make suggestions, but the decision of what Product Backlog can constitute the Sprint is the responsibility of the Product Owner.
- The Team is responsible for determining how much of the Product Backlog that the Product Owner wants worked on the Team will attempt to do during the Sprint.
- Time-boxing the first segment to 4 hours means that this is all of the time that is available for analyzing the Product Backlog. Further analysis must be performed during the Sprint. Large-grained, high-priority Product Backlog with imprecise estimates might not be thoroughly understood during this part of the Sprint planning meeting and might result in the Team not being able to complete all of the Product Backlog that it selects.
- The second segment of the Sprint Planning meeting occurs immediately after the first segment and is also time-boxed to 4 hours.
- The Product Owner must be available to the Team during the second segment to answer questions that the Team might have about the Product Backlog.
- It is up to the Team, acting solely on its own and without any direction from outside the Team, to figure out during the second segment how it will turn the selected

Product Backlog into an increment of potentially shippable product functionality. No one else is allowed to do anything but observe or answer questions seeking further information.

- The output of the second segment of the Sprint planning meeting is a list, called the Sprint Backlog, of tasks, task estimates, and assignments that will start the Team on the work of developing the functionality. The task list might not be complete, but it must be complete enough to reflect mutual commitment on the part of all Team members and to carry them through the first part of the Sprint, while the Team devises more tasks in the Sprint Backlog.

Team LiB

◀ PREVIOUS

NEXT ▶

Daily Scrum Meeting

The Daily Scrum meeting is time-boxed to 15 minutes regardless of the number of Team members.

- Hold the Daily Scrum in the same place at the same time every work day. The Daily Scrum is best held first thing in the day so that the first thing Team members do on arriving at work is think of what they did the day before and what they plan to do today.
- All Team members are required to attend. If for some reason a Team member can't attend in person, the absent member must either attend by telephone or by having another Team member report on the absent member's status.
- Team members must be prompt. The ScrumMaster starts the meeting at the appointed time, regardless of who is present. Any members who are late pay \$1 to the ScrumMaster immediately.
- The ScrumMaster begins the meeting by starting with the person immediately to his or her left and proceeding counterclockwise around the room until everyone has reported.
- Each Team member should respond to three questions only:
 - What have you done since the last Daily Scrum regarding this project?
 - What will you do between now and the next Daily Scrum meeting regarding this project?
 - What impedes you from performing your work as effectively as possible?
- Team members should not digress beyond answering these three questions into issues, designs, discussion of problems, or gossip. The ScrumMaster is responsible for moving the reporting along briskly, from person to person.
- During the Daily Scrum, only one person talks at a time. That person is the one who is reporting his or her status. Everyone else listens. There are no side conversations.
- When a Team member reports something that is of interest to other Team members or needs the assistance of other Team members, any Team member can immediately arrange for all interested parties to get together after the Daily Scrum to set up a meeting.
- Chickens are not allowed to talk, make observations, make faces, or otherwise make their presence in the Daily Scrum meeting obtrusive.
- Chickens stand on the periphery of the Team so as not to interfere with the meeting.
- If too many chickens attend the meeting, the ScrumMaster can limit attendance so that the meeting can remain orderly and focused.

- Chickens are not allowed to talk with Team members after the meeting for clarification or to provide advice or instructions.
- Pigs or chickens who cannot or will not conform to the above rules can be excluded from the meeting (chickens) or removed from the Team (pigs).

Team LiB

◀ PREVIOUS NEXT ▶

Sprint

The Sprint is time-boxed to 30 consecutive calendar days. Aside from other factors, this is the amount of time required for a Team to build something of significant interest to the Product Owner and stakeholders and bring it to a state where it is potentially shippable. This is also the maximum time that can be allocated without the Team doing so much work that it requires artifacts and documentation to support its thought processes. It is also the maximum time that most stakeholders will wait without losing interest in the Team's progress and without losing their belief that the Team is doing something meaningful for them.

- The Team can seek outside advice, help, information, and support during the Sprint.
- No one can provide advice, instructions, commentary, or direction to the Team during the Sprint. The Team is utterly self-managing.
- The Team commits to Product Backlog during the Sprint planning meeting. No one is allowed to change this Product Backlog during the Sprint. The Product Backlog is frozen until the end of the Sprint.
- If the Sprint proves to be not viable, the ScrumMaster can abnormally terminate the Sprint and initiate a new Sprint planning meeting to initiate the next Sprint. The ScrumMaster can make this change of his or her own accord or as requested by the Team or the Product Owner. The Sprint can prove to be not viable if the technology proves unworkable, if the business conditions change so that the Sprint will not be of value to the business, or if the Team is interfered with during the Sprint by anyone outside the Team.
- If the Team feels itself unable to complete all of the committed Product Backlog during the Sprint, it can consult with the Product Owner on which items to remove from the current Sprint. If so many items require removal that the Sprint has lost its value and meaning, the ScrumMaster can abnormally terminate the Sprint, as previously stated.
- If the Team determines that it can address more Product Backlog during the Sprint than it selected during the Sprint planning meeting, it can consult with the Product Owner on which additional Product Backlog items can be added to the Sprint.
- The Team members have two administrative responsibilities during the Sprint: they are to attend the Daily Scrum meeting, and they are to keep the Sprint Backlog up-to-date and available in a public folder on a public server, visible to all. New tasks must be added to the Sprint Backlog as they are conceived, and the running, day-to-day estimated hours remaining for each task must be kept up-to-date.

Sprint Review Meeting

The Sprint review meeting is time-boxed to 4 hours.

- The Team should not spend more than 1 hour preparing for the Sprint review.
- The purpose of the Sprint review is for the Team to present to the Product Owner and stakeholders functionality that is done. Although the meaning of “done” can vary from organization to organization, it usually means that the functionality is completely engineered and could be potentially shipped or implemented. If “done” has another meaning, make sure that the Product Owner and stakeholders understand it.
- Functionality that isn’t “done” cannot be presented.
- Artifacts that aren’t functionality cannot be presented except when used in support of understanding the demonstrated functionality. Artifacts cannot be shown as work products, and their use must be minimized to avoid confusing stakeholders or requiring them to understand how systems development works.
- Functionality should be presented on the Team member workstations and executed from the server closest to production—usually a quality assurance (QA) environment server.
- The Sprint review starts with a Team member presenting the Sprint goal, the Product Backlog committed to, and the Product Backlog completed. Different Team members can then discuss what went well and what didn’t go well in the Sprint.
- The majority of the Sprint review is spent with Team members presenting functionality, answering stakeholder questions regarding the presentation, and noting changes that are desired.
- At the end of the presentations, the stakeholders are polled, one by one, to get their impressions, any desired changes, and the priority of these changes.
- The Product Owner discusses with the stakeholders and the Team potential rearrangement of the Product Backlog based on the feedback.
- Stakeholders are free to voice any comments, observations, or criticisms regarding the increment of potentially shippable product functionality between presentations.
- Stakeholders can identify functionality that wasn’t delivered or wasn’t delivered as expected and request that such functionality be placed in the Product Backlog for prioritization.
- Stakeholders can identify any new functionality that occurs to them as they view the presentation and request that the functionality be added to the Product Backlog for prioritization.
- The ScrumMaster should attempt to determine the number of people who expect to attend the Sprint review meeting and set up the meeting to accommodate them.
- At the end of the Sprint review, the ScrumMaster announces the place and date of the next Sprint review to the Product Owner and all stakeholders.

Team LiB

◀ PREVIOUS

NEXT ▶

Team LiB

◀ PREVIOUS NEXT ▶

Sprint Retrospective Meeting

The Sprint retrospective meeting is time-boxed to 3 hours.

- It is attended only by the Team, the ScrumMaster, and the Product Owner. The Product Owner is optional.
- Start the meeting by having all Team members answer two questions:
 - What went well during the last Sprint?
 - What could be improved in the next Sprint?
- The ScrumMaster writes down the Team's answers in summary form.
- The Team prioritizes in which order it wants to talk about the potential improvements.
- The ScrumMaster is not at this meeting to provide answers, but to facilitate the Team's search for better ways for the Scrum process to work for it.
- Actionable items that can be added to the next Sprint should be devised as high-priority nonfunctional Product Backlog. Retrospectives that don't result in change are sterile and frustrating.

Team LiB

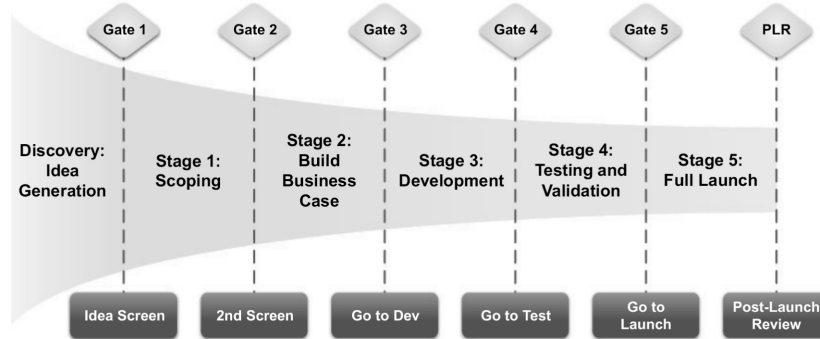
◀ PREVIOUS NEXT ▶

Winning at New Products

By Robert G. Cooper

Chapter 4: The Stage-Gate Idea-to-Launch System

Figure 4.5: *Stage-Gate* Is a Conceptual and Operational Roadmap from Idea Through to Launch and Beyond



Artwork courtesy of Dave Caissy

Suggestion: The five decision rules outlined above apply to almost any high-risk situation. Does your company follow them in its day-to-day management practices? Review your firm’s new-product practices, perhaps using an actual case, and assess whether your management group is handling risk appropriately.

Seven best-practice goals together with the key success drivers from the last two chapters have been fashioned into the *Stage-Gate system*—a *conceptual and operational model* for moving a new-product project from idea to launch.

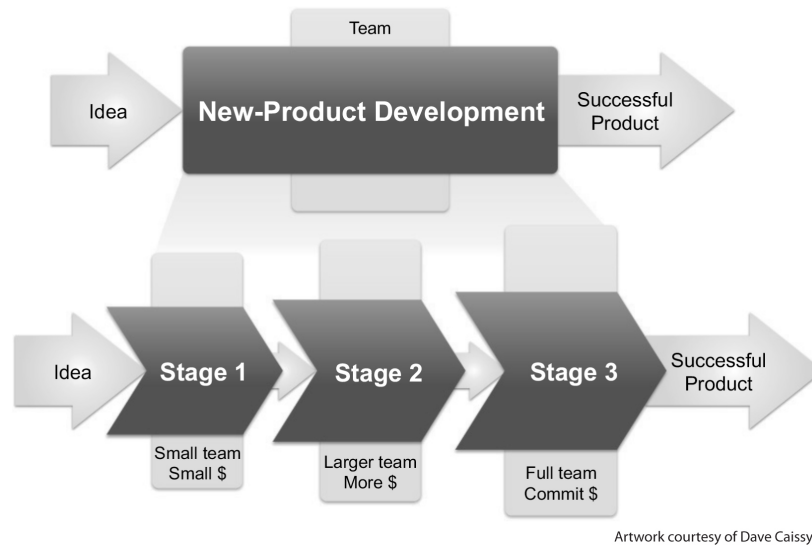
ABEST-PRACTICE NEW-PRODUCT SYSTEM

The Structure of the Stage-Gate System

The seven key goals and the five gambling rules, together with the success drivers from the last two chapters, have been fashioned into a best-practice *Stage-Gate* system—a *conceptual and operational model* for moving a new-product project from idea to launch.¹¹ This *Stage-Gate* system is a playbook or blueprint for managing product innovation to improve effectiveness and efficiency—see [Figure 4.5](#).

Stage-Gate is elegantly simple, and makes a lot of sense: a series of information-gathering stages followed by decision-making gates. *Stage-Gate* treats product innovation as a process, and any process can be modeled and made better and more efficient. Instead of viewing product development as a black box—ideas go in and commercially ready products come out (or a black hole—ideas go in and nothing comes out)—*Stage-Gate* breaks the innovation process into a predetermined set of manageable, discrete stages (see [Figure 4.6](#)). Each stage consists of a set of prescribed, cross-functional, and parallel activities:

Figure 4.6: The *Stage-Gate* System Breaks Product Development Into a Series of Manageable Stages with Increasing Resource Commitments



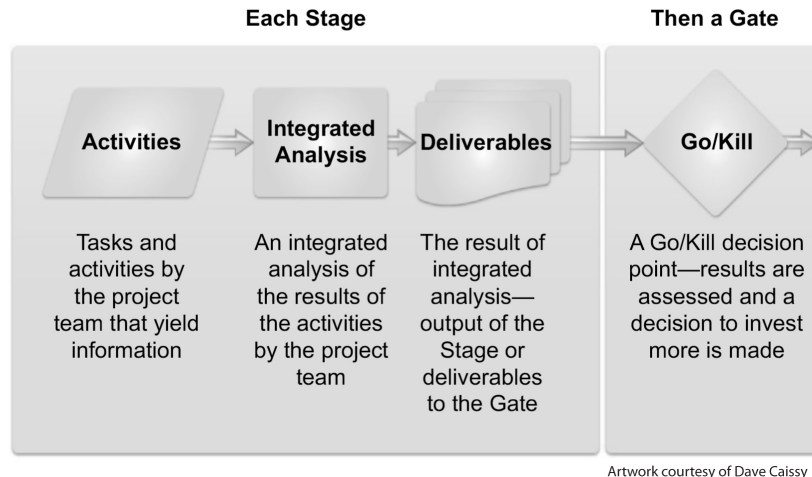
- Teams of people undertaking some vital activities to gather some data,
- followed by data analysis and interpretation,
- to create the key deliverables (information),
- on the basis of which senior management—the resource owners—make the Go or Kill decision.

And the process repeats (see [Figure 4.7](#)). The entrance to (or exit from) each stage is a gate: The gates control the process and serve as the quality control and Go/Kill checkpoints. This stage-and-gate format leads to the name “*Stage-Gate* system.”

The Stages

Each stage is designed to gather information needed to move the project to the next gate or decision point. Different types of information—market, technical, operations—is important, and so the work within each stage is cross-functional: There is no “R&D stage” or “marketing stage.” And each stage is defined by the activities within it—a set of parallel and cross-functional tasks that incorporate best practices, for example:

Figure 4.7: *Stage-Gate* Consists of a Set of Information-Gathering Stages Followed by Go/Kill Decision Gates



- the ten vital market-related actions and the key homework activities listed above in Tables 4.1 and 4.2;
- some of the success drivers, for example, quality of execution and sharp, early product definition; and
- the seven goals, for example, the quest for a superior, differentiated product, that we saw earlier in this and the last two chapters.

Some of these activities are mandatory; others are merely prescribed and highly recommended—*Stage-Gate is a guide, not a rule book*. These stage-activities are designed to gather information and drive uncertainties down. And each stage typically costs more than the preceding one: The process is an incremental commitment one, like buying a series of options.

The general flow of the typical *Stage-Gate* system is shown pictorially in [Figure 4.8](#). The stages are:

0. *Discovery*: prework designed to discover and uncover opportunities and generate ideas.

Figure 4.8: The Five Stages in the Typical Idea-to-Launch Stage-Gate System from Discovery (Ideation) Through to Launch

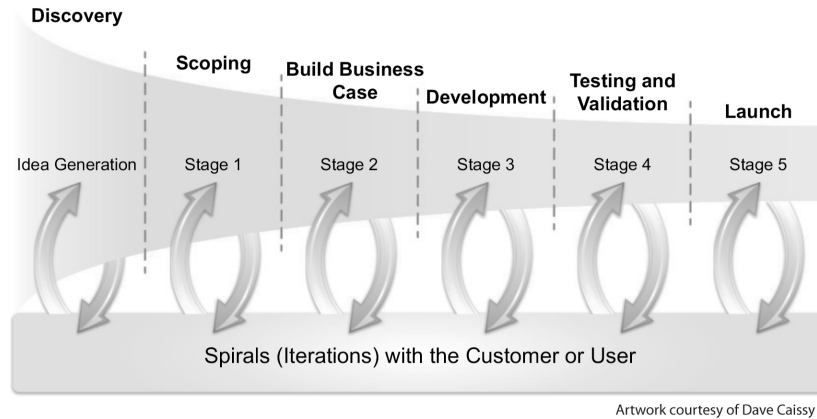
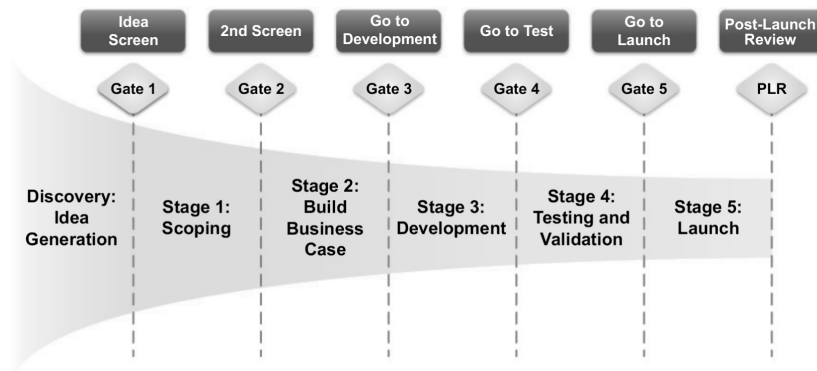


Figure 4.9 Each Stage Is Preceded by a Gate—the Decision or Go/Kill Points in the System



Gates are where projects get resources and are prioritized—
they get on management's radar screen

Gates are also the quality-control checkpoints in the system:

- Are you doing the project right?
- Are you doing the right project?

Artwork courtesy of Dave Caissy

1. *Scoping*: a quick, preliminary investigation and scoping of the project—largely desk research.
2. *Build the Business Case*: a much more detailed investigation involving primary research—both market and technical—leading to a Business Case, including product and project definition, project justification, and a forward plan (or action plan).
3. *Development*: the actual detailed design and development of the new product, and the design of the operations or production process.
4. *Testing and Validation*: market tests or trials, technical tests, and operations trials to verify and validate the proposed new product, and its marketing and production/operations.

5. *Launch*: commercialization—beginning of full operations or production, marketing, and selling.

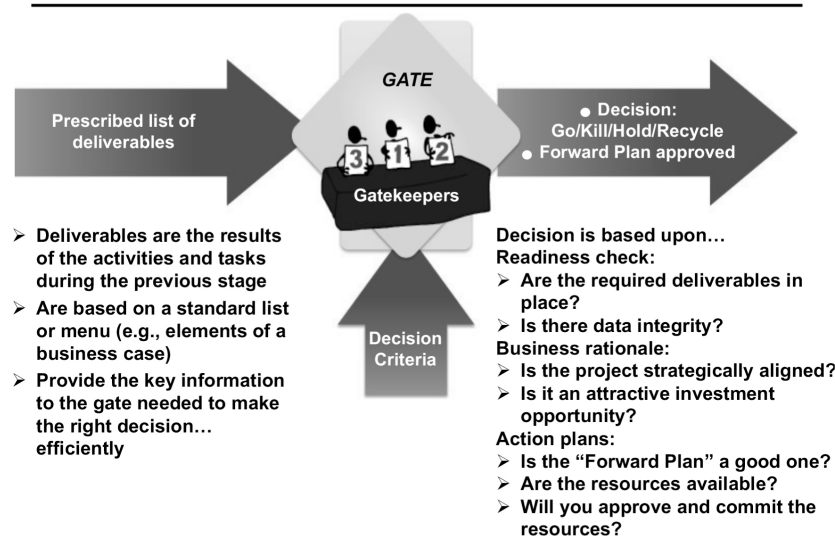
The Gates

Preceding each stage is a gate or a *Go/Kill decision point*—see [Figure 4.9.*](#) The gates are the huddles on the football field: They are the points where the team converges and where all new information is reviewed. Gates serve as quality-control checkpoints, as Go/Kill and prioritization decision points, and as points where the path forward for the next play or stage of the process is agreed to.

The structure of each gate is similar ([Figure 4.10](#)). Gates consist of:

1. A set of required *gate deliverables*: what the project leader and team must bring to the decision point (e.g., the results of a set of completed tasks). These deliverables are visible, are based on a standard menu for each gate, and are decided at the output of the previous gate. Management’s expectations for project teams are thus made very clear. Deliverables are usually defined in the format of *templates*.
2. *Gate criteria* against which the project is judged: These can include “must meet” or knock out questions (a checklist) designed to weed out misfit or “nonstarter” projects quickly, for example:
 - Is the proposed project within our business or strategic mandate?
 - Does it meet our EH&S policies (environmental, health, and safety)?

Figure 4.10: Gates Have a Common Format—Inputs, Criteria, and Outputs



Artwork courtesy of Dave Caissy

There are also scorecard criteria (“should meet” criteria or desirable factors),

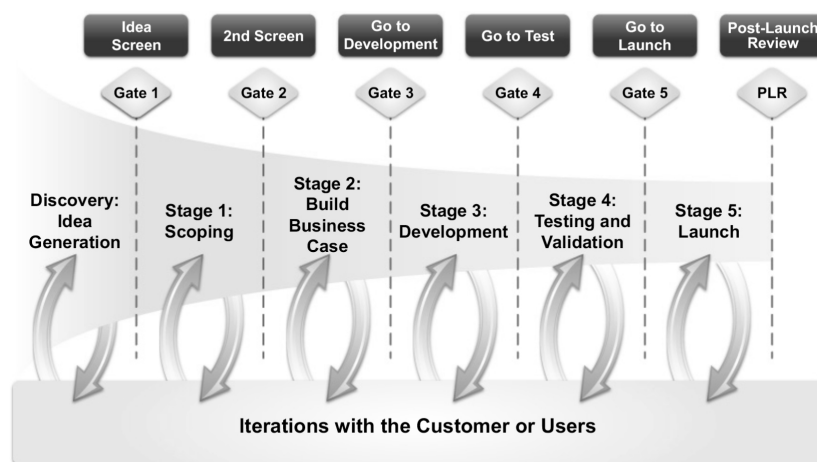
which are scored and added (a point count system), which are also used to make Go/Kill decisions as well as to prioritize projects, for example:

- the strength of the value proposition or product's competitive advantage,
- ability to leverage core competencies in this project,
- relative market attractiveness, and
- size of the financial return versus the risk.

3. Defined *gate outputs*: for example, a decision, usually one of four possible outcomes: *Go*, *Kill*, *Hold*, or *Recycle*; and in the event of a Go decision, an approved action plan for the next stage complete with resource commitments (people required, money, and person-days), an agreed timeline, and a list of deliverables and date for the next gate.

Gates are usually staffed by senior managers from different functions, who own the resources required by the project leader and team for the next stage. They are called the *gatekeepers*, and are a predefined group for each of the five gates. For example, for larger projects, Gates 3, 4, and 5 are often staffed by the leadership team of the business—the head of the business and the heads of marketing/sales, technology, operations, and finance. Earlier gates are often staffed by middle management, as the resource commitments are less.

Figure 4.11: This Five-Stage Idea-to-Launch Stage-Gate® System Is for Major and Higher Risk New-Product Projects



For lower-risk and smaller development projects,
use an abbreviated version: two to three Gates

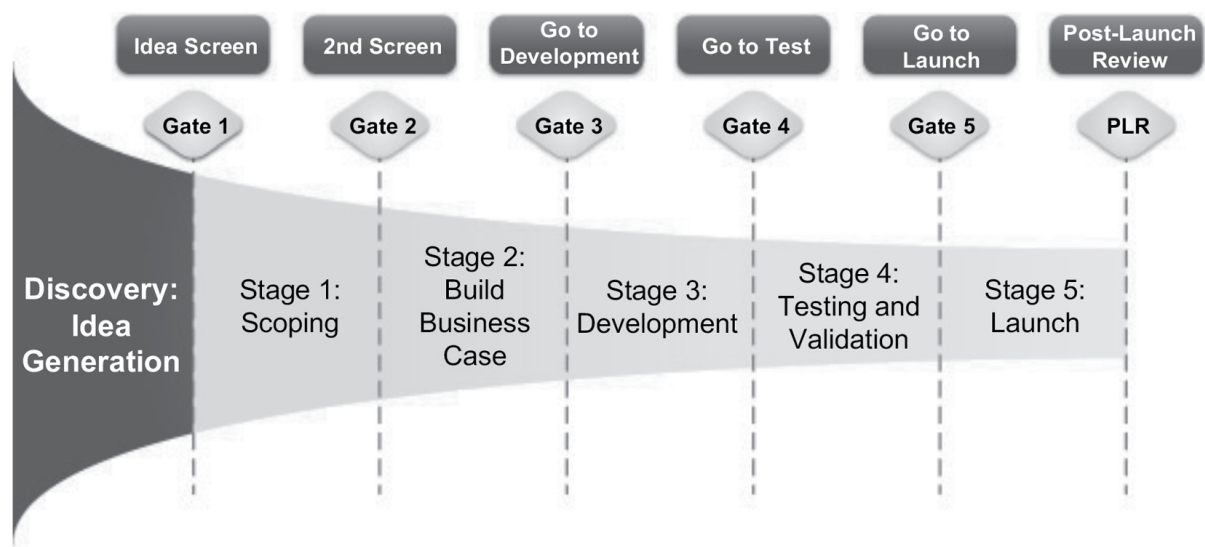
Artwork courtesy of Dave Caissy

Gates feature deliverables (what the project team delivers to the gate); criteria to make the Go/Kill and prioritization decisions (often in the

form of a scorecard); and gate outputs (the Go/Kill/Hold/Recycle decision and what each decision translates into in practice).

AN OVERVIEW OF THE STAGE-GATE SYSTEM

Now for a bird's-eye look at the *Stage-Gate* system—an overview of what's involved at each stage and gate. In later chapters, we'll lower the microscope on the Discovery stage, or how to generate breakthrough ideas ([Chapter 7](#)). [Chapters 8](#) and [9](#) take a close look at how to design and operate gates or decision points. And [Chapters 5](#) and [6](#) look at next generation versions of the gating system. But for now, let's just have a *quick walk-through* of the *basic model* designed for larger new-product projects, which you can follow stage by stage in [Figure 4.11](#).



Discovery or Ideation

Ideas are the feedstock or trigger to the process, and they make or break the system. Don't expect a superb new-product process to overcome a deficiency in good new-product ideas. The need for great ideas coupled with the high attrition rate of ideas means that the idea-generation stage is pivotal: You need great ideas and lots of them.

Many companies consider ideation so important that they handle this as a formal stage in the process, the one we call Discovery—that is, they build in a *defined, proactive idea-generation and capture system*. Many activities can be part of the Discovery stage in order to stimulate the creation of great new-product ideas. Such activities include: undertaking fundamental but directed technical research; seeking new technological possibilities; working with lead or innovative users; utilizing VoC research to capture unarticulated needs and customer problems; competitive analysis and inverse brainstorming of competitive products; installing an idea-suggestion scheme to stimulate ideas from your own employees; scanning the outside world and employing

Hybrid Project Management

By Mark Tolbert and Susan Parente

Chapter 1: Hybrid Projects: The Need
to Be Open to Different Project
Management Methodologies

the necessary funding for this opportunity. Bottom line, it's much easier today for people to form new companies to go after opportunities, and people are doing exactly that today. This has raised the level of competition in major ways. A number of different writers say that the Internet has brought about as profound a change in our world as the Gutenberg printing press did in the 15th century. So, in short, we're living in this age of accelerations with tremendous change and a heightened level of competition.

Why does this matter to us as PMs? Why do we care about all this volatility, difficult marketplaces, and the challenges our companies are facing? Of course, it's simply because we are helping our companies survive in these difficult times. We are managing change for them, and we are helping our companies create the new products and solutions that will resonate with customers. We are drivers of change in this new world! Therefore, our companies need us to be very good at managing projects and to be up-to-speed with the latest methodologies. So, what project management methodology is best suited for coping with change and handling volatility? Of course, that's an easy answer; the answer is Agile! But we shouldn't discard the tried and true—the traditional waterfall approaches. As we'll see, this methodology also has an important role in many projects. We need to be open-minded and know when and where to use the different approaches and the strengths and weaknesses of the different approaches.

The Roots of Modern Project Management and the Case for Traditional Project Management

From what industry did project management arise and who wrote the first books and anthologies on project management? It was the construction and engineering industry that provided the foundations of our modern project management practices. The first authors of college textbooks on project management were engineering professors: people like Clelland and King (1968) and Harold Kerzner (1979).

Some people might say that modern project management starts with Henry Gantt and Gantt charts in 1910. These were used successfully with the Hoover Dam project in the 1930s. Other people point to the 1950s

with the advent of Program Evaluation and Review Technique (PERT), which was used with the Navy Polaris Submarine program, and with critical path method (CPM), devised in the late 1950s at DuPont. Also, much of the traditional project management approach comes to us from the 1960s and the Department of Defense (DoD). Much of what we teach in our Project Management Professional (PMP)[®] Prep classes is based on the core principals developed in DoD, especially EVM and the WBS. The WBS is a key part of Earned Value (EV) and is very close to the heart of the traditional PM. The WBS drives all planning, or is the “cornerstone of all planning.” It drives scheduling, cost estimating, and budgeting, determining quality acceptance values, resource estimating, identifying risks, and deciding what we want to outsource. Of course, we also have the 100 percent rule: “100 percent of the project must be in the WBS! If it’s not in the WBS, it’s not in the project!”

What is the lifecycle model that is very closely tied to traditional project management? The “waterfall lifecycle approach” and the use of predictive planning. This model entails:

- Performing exhaustive, detailed planning for all phases of the project at the beginning (“The devil is in the details!”). Therefore, don’t leave any stones unturned; figure out all requirements, in detail, in your planning phases at the beginning of the project. Plan everything so well from the beginning that change won’t be needed! As Harold Kerzner says in his textbook on project management,⁴ do planning so well that scope changes are kept to a minimum.
- Hierarchically decomposing your original business requirements and functional requirements to a very detailed level using a WBS.
- Obtaining accurate, detailed estimates for time and cost at the work-package level using techniques such as bottom-up estimating, parametric estimating, and PERT three-point estimating.

⁴ *Project Management—A Systems Approach to Planning, Scheduling, and Controlling* by Harold Kerzner, Ph.D., p. 6.

- Measuring performance during the project using EVM.
- Measuring performance against all three baselines (scope, schedule, and cost) using EVM, and using *Quantitative Risk Analysis* techniques to more accurately define contingency reserves to include in the cost baseline and schedule baseline.

If you're building a new bridge or building, you probably want to use this waterfall approach. You had better get very detailed, accurate blueprints created before you start construction. Then, you will need very accurate baselines with which to measure your performance. Detailed blueprints (and a WBS) will allow you to create an accurate schedule and budget, so you can measure your performance against time and cost. Ideally, you will use EVM to do this. In EVM, they like to say, "To do project management properly, you need to have *defined scope*." If you have this, you can define all three key baselines: scope, schedule, and cost.

EVM really works hand-in-hand with traditional project management and a waterfall approach. For EVM to work, we need to have very accurate baselines and a very accurate BAC (budget at completion). In an excellent book on EV written by Fleming and Koppelman, *Simple Earned Value on All Projects—(Simplified Translations of the 27 EVM Criteria)*, they state the most important of all the 27 criteria is the very first one. Here is their translation:

Step 1: To the extent possible, you must define the full scope of the project.—(Equates to EVM Criterion #1)

- This is perhaps the most important requirement for implementing earned value, and perhaps the most difficult to achieve. Management of certain types of projects, notably software, often give up at this point and refuse to go further. They often relinquish (p. 160).
They go further on p. 178:
- The first group of criteria deals with the requirement for any new project to be completely defined, and planned, prior to starting performance of the work. Today, we would typically call this effort defining the scope of the project. Think about

it: Earned Value Measurement cannot take place without some definition of what constitutes 100 percent of the project.

Why is it so crucial in construction and engineering to first obtain the full scope of the entire project—all the requirements spelled out in detail—before we start construction? The answer is pretty obvious. If there's a mistake in our blueprints and construction begins; and, if the mistake in the blueprints is not discovered until weeks have gone by, and a lot of construction work has been done, all that engineering and construction work might have to be torn up and redone. Of course, that is going to be very, very expensive. Someone's probably going to lose their job!

My youngest son, Michael, works for a design/build mechanical firm using 3D CAD (computer-aided design). He's creating the blueprints for all the mechanical and plumbing work in large high-rise apartment buildings. He's told me, "Dad, if my blueprints for the mechanical and plumbing design in the building are off by even a quarter of an inch, then there will be hell to pay." Predictive planning must be done, and every detail nailed down, too. Michael's BIM (Building Information Modeling) team must coordinate everything at the start of a construction project to collaborate with the engineering team and make sure each floor of the building is "clash free" (virtually) before construction starts. This is necessary to avoid expensive change orders that could potentially happen in the field. For example, suppose the BIM team didn't realize there was a 24×48 beam blocking a 22×14 duct going into the lobby area! "The Devil is in the details!"

Is predictive planning and this attention to detail only critical in construction and engineering projects where the cost of change once we started construction could be very, very high? If we're doing a software application, and the change will just involve moving bits around and recompiling programs, do we still have the same urgency to get all our requirements totally correct at the beginning? Is it even possible to do that?

On construction projects, it's usually the case that the customer knows much more precisely what they want going into the project than they would on a typical software project, or "knowledge work" project. Perhaps it is a new office building. In most cases, the customer will know

exactly what they can afford, when the building needs to be completed, and the other requirements: number of offices, conference rooms, size of the lobby area, number of floors and footprint, power and HVAC requirements, networking requirements, architectural style, and so on. The architect will create detailed blueprints and models that the customer will review and accept before construction begins. Or ... perhaps it's a home kitchen remodeling project. The key stakeholder—the wife—will choose exactly what appliances she wants, the cabinets, and counter-tops. Perhaps a general contractor will be engaged, and after design meetings with the homeowners, they will create blueprints or CAD drawings of the new kitchen that the homeowners will accept or modify. Again, the customer will determine, in detail, all the requirements before the “demolition” and “remodeling” phases begin. Therefore, on these types of projects, it's a much more straightforward process to obtain all the requirements upfront at the beginning of the project.

These types of projects are much more conducive to using a fixed-price contract. Once given the detailed requirements, the contractor will be able to determine their costs precisely and generate a fixed-price bid. (Fixed-price contracts and other contracts are presented in more detail in Chapter 2—“Agile Contracts: Can Agile Be Used with Fixed-Price Contracts?”)

Risk #1: The Number One Risk on Projects!

There are several other reasons the traditional “waterfall” methodology often makes sense. What is the principal cause of failure on projects and, therefore, our “Number One Risk?” It's having a poorly written scope statement: a vague, ambiguous scope statement. If we allow that to happen, there will be no hope for this project. In a key process—*Validate Scope*—defined in the Scope Management KA, *Project Management Body of Knowledge (PMBOK®) Guide*,⁵ we bring in the customer to inspect deliverables that we've created and obtain their formal acceptance. If our scope statement is vague or ambiguous, and the acceptance criteria in

⁵ Project Management Institute, *A Guide to the Project Management Body of Knowledge*, Sixth Edition, Project Management Institute, Inc., 2017.

the scope statement are poorly written—not “SMART”—specific, measurable, agreed, realistic, and timebound—how is this meeting with the customer going to go? Of course, not well at all! What are we likely to hear? “I’m sorry, but this isn’t exactly what I’m looking for.” ... and, of course ... “Weren’t you listening? I absolutely need to have these other requirements too!” ... etc. And, if our scope statement is vague or ambiguous and if the acceptance criteria for this deliverable were poorly written, then we’re going to have to add in these extra requirements at our own expense. This leads to what we hate most—Scope Creep!⁶ Scope Creep is “The uncontrolled expansion of project scope or product scope without making adjustments to time, cost and resources,”⁷ which leads to schedule delays, budget overruns, customer satisfaction issues, and senior management satisfaction issues with the project. And, very likely, a new PM! Therefore, follow all the *PMBOK® Guide* planning processes in Scope Management very well: *Plan Scope Management*; *Collect Requirements*; *Define Scope*; *Create WBS*. Following these processes ensures you are on the same page with the customer; all the requirements have been captured and the acceptance criteria for the deliverables are “SMART.”

This leads us to a discussion of why the job of the PM is a very difficult job. I like to say, “Project management is not for sissies!” Project management is essentially all about people skills or soft skills. It is not really about understanding the technical intricacies of scheduling tools like Microsoft Project, Primavera, or Artemis (e.g., understanding how to do a forward pass and backward pass; understanding the difference between total float and free float). It is not about understanding the arcane tools and techniques mentioned for *Quantitative Risk Analysis*: EMV (expected monetary value) or Monte Carlo simulation or decision trees or a sensitivity analysis. It is not about the “IQ (Intellectual Quotient) part of the equation”; it is about the “EQ (Emotional Quotient) part of the equation” or

⁶ To be technically correct, “Scope Creep” is not a risk, it’s the effect or outcome if we do not handle risks in Scope Management well. Once a risk occurs, it’s no longer a risk; it’s an “issue.” Risks are all about uncertainty and probability.

⁷ Project Management Institute, *A Guide to the Project Management Body of Knowledge, (PMBOK® Guide)*, Sixth Edition, Project Management Institute, 2017, Page 168.

emotional intelligence. What is the key part of emotional intelligence or the key part of people skills and soft skills? Clearly, the answer is good communication skills. What is the essence of having good communication skills? (Something that technical people like me have a lot of difficulty with!) Good listening skills. All good communicators are necessarily very good listeners.

In the *PMBOK® Guide*, PMI® does not put any priorities on the processes in the different Knowledge Areas. But I will go out on a limb and do that! I think the highest priority Knowledge Areas are *Scope Management*; *Communications Management*; and *Stakeholder Management*. Success on our projects directly depends on how well we perform key processes in these Knowledge Areas, and these Knowledge Areas all really overlap one another (Figure 1.1).

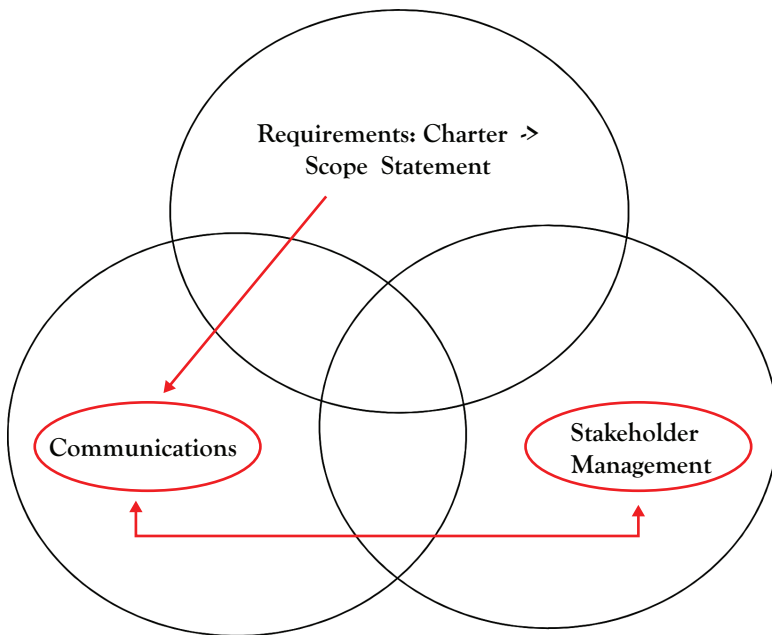


Figure 1.1 Foundational KAs (processes)

1. As we said, our principal risk involves doing Scope Management well and this starts with doing the requirements gathering process well.
 - *Don't miss any requirements.*
 - *Don't misunderstand the requirements.*

- *Don't misunderstand the complexity of the requirements:
Decompose requirements properly.*
 - *Don't miss any key stakeholders in obtaining requirements.*
2. Successfully obtaining requirements involves communicating very well with the customer and other key stakeholders. This includes listening! Follow well-accepted practices like JAD (joint application design) and QFD (quality function deployment). In essence, get those engineers out of their offices to go meet with the customer. Have productive conversations with the customer to truly understand what they are seeking. Don't assume that we know best what the customer really wants! Get out of our offices, follow the customer around to see what they're really needing. Even better, try out their job for part of the day, so we truly understand what they need.
 3. Let's assume that we do the requirements gathering process very well; we erred on the side of inclusiveness, we've done brainstorming, we've been creative, and we correctly captured all the different stakeholder requirements. Are we now in good shape? Are we going to be successful? No, not necessarily! This is where the hard part of project management comes in. Are the stakeholders all in agreement for the requirements for the project? Of course not! The requirements they want are all over the map. They have different needs and different wants. Somehow, somehow, we have to get them onto the same page. If we don't do that, this project is surely going to fail, and who's going to take the blame? Not these stakeholders who are giving us all these inconsistent requirements. No, you will take the blame (the PM).

Many of the PMP® Prep books completely misunderstand this key point about what is happening as we move from *Collect Requirements* to *Define Scope*. The *PMBOK® Guide* doesn't even emphasize this enough! Many people think that as we move into *Define Scope*, we are just digging into more and more detail. That is not the key purpose of *Define Scope*. No, the key purpose is to get everyone onto the same page; “define boundaries,” and “define exclusions.” This is the hard part of project management. No matter what powers we have as PMs—we could be that very empowered PM in a strong matrix organization that we assume is the default for the PMP®

exam—we will never have enough power to legislate what will be in scope and what will not be in scope. No matter how powerful we are, we are dealing with stakeholders who are much more powerful than we are: the sponsor and other senior managers in our own organization, the customer, even stakeholders in other organizations (e.g., federal regulatory organizations).

Nonetheless, for this project to succeed, we must get everybody onto the same page; we must define the boundaries and the exclusions for the project. If this doesn't happen, this project will fail. I've been to many dinner meetings for the Washington, DC PMI Chapter (PMIWDC) over the past 20+ years, and I've heard many excellent presentations about projects where things went dreadfully wrong. In almost all cases (F-22, Affordable Healthcare Act website project, etc.), these projects failed because this problem was not correctly handled. So, we must obtain requirements correctly and create a well-written scope statement with SMART acceptance criteria. To do that, we must do communications very well, but we must also do *Stakeholder Management* very well to define these boundaries and exclusions.

By the way, who has most of the risk in this situation? The sponsor? The customer? The senior managers who gave us the incompatible requirements? No, the PM has the risk! If we fail to get everyone on the same page and proceed trying to run this project, it is surely going to fail. When that happens, who will take the blame, and who will be looking for a new job? We will: the PM!

4. Criterion number one, defined in the most current EVM specification, is directly addressing this point, “to the extent possible you must define the full scope of the project!”

But, let's suppose we do all this correctly; we do correctly follow the planning processes in Scope Management defined in the *PMBOK® Guide*, we do get boundaries and exclusions defined, and do get everyone “onto the same page.” Does that now guarantee success for the project? No, it does not!

The Case for Agile

Suppose the customer doesn't know exactly what they want at the beginning of the project, and they want a new solution created for them. They know high-level business requirements or functional requirements, but they don't know the exact detailed engineering solution. That needs to be explored and discovered, and it's understood there's going to be some trial and error in discovering the best engineering solution. But, instead of using an iterative approach, we follow the guidance of Fleming and Koppelman, and we do our best at the beginning of the project to uncover in a detailed way all the scope requirements. It is 1978, and we are using a waterfall approach for developing this application. We are going to do predictive planning! (Waterfall approaches using "predictive planning" were typically used for developing software applications in the 1970s and 1980s. Some people are still using this type of approach for software applications today!) A simplified view of the waterfall lifecycle model we used might look like the one in Figure 1.2.

We spent several months interviewing all the different stakeholders on the project, making sure that we didn't miss anyone or leave anyone out. We thoroughly captured all of their requirements for this new application: layouts of reports they wanted to see, the graphical user interface (GUI) for input screens, file layouts of other input files, expected response times and throughput times, and other details. We created the user

Organize the project in phases. Phases are often organized in a sequential manner (or "Waterfall" structure.)

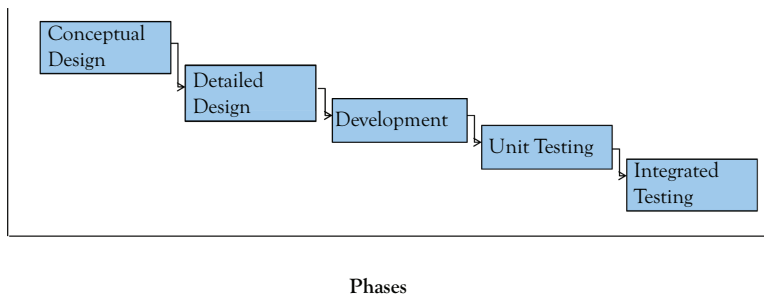


Figure 1.2 Traditional project lifecycle structure

requirements documentation (the functional specification). We obtained all the necessary approvals, and had a bow tied around these documents.

We then turned over these documents to the systems analysts, who did the detailed design. They hierarchically decomposed the application: from the application down to programs, from the programs down to modules, from modules down to subroutines, and from subroutines down to functions, and so forth. They defined the interfaces between each level and the parameters that would be passed. They defined all the file layouts. This also took several months to accomplish. Again, the necessary approvals were obtained on the subsystem specification, or the system architecture document. Now, the system architecture documentation was turned over to the Cobol and Fortran programmers, who did the coding and development. How long did that take? An average application at the time of 1978 was usually more than 100,000 lines of code, and it typically took more than a year to do all the coding and unit testing. Then the programmers brought all their code together to do integrated testing, and this could also take more than a month or two.

Finally, that glorious day arrived: UAT! (User acceptance testing). How did UAT go? Not well at all! Why? Well, how long was it before the customer really got to see the running application? It was 18 months or longer! What's happened in that year and a half? The world has changed! By the time of UAT, there were much better ways to do things; better ways to index records to provide faster response times; and better GUIs and screen designs for inputting data and reviewing data. The customer wanted these new technologies, and was not happy with an application that had been designed 18 months or more earlier. Even more fundamentally, it's not until the customer uses the application, sees real response times and throughput times, plays with the screens and sees how everything works that they realize what they really needed in the first place. So, they are not happy at all!

Therefore, Fleming and Koppelman were wrong! It's not that as software PMs we gave up too soon and didn't try hard enough to determine the real requirements the customer needed; it just wasn't possible to do that. The world was changing too quickly, and at the time of obtaining the user requirements, we could not see, and the customer could not see, how things would change and what detailed engineering solution would

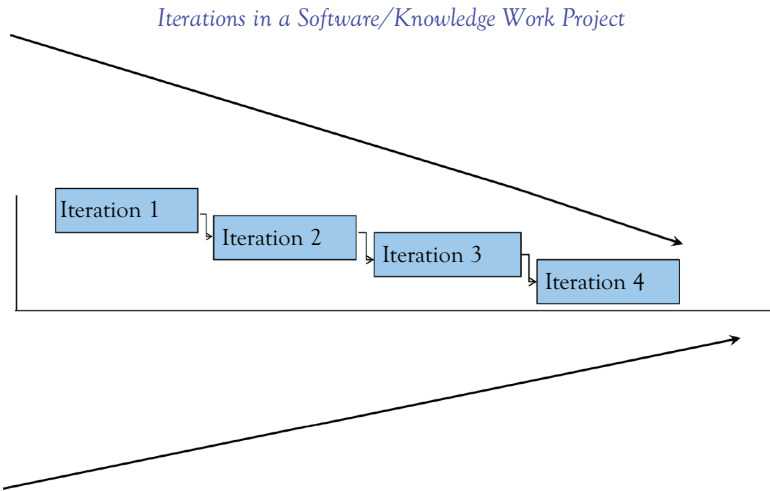


Figure 1.3 The “Cone of Uncertainty”

work best. In our world today—Tom Friedman’s *Age of Accelerations*—as PMs, we are dealing much more with the “Cone of Uncertainty” (Figure 1.3).

In the first iterations of the project, it’s impossible to predict accurately exactly what detailed design solution will work best for the project at its end-date—perhaps one year out, or longer. The world is changing very quickly, and new technologies and solutions will become available, and these will likely change the final engineering solution the team decides upon. It’s also impossible at the beginning to get accurate forecasts of time and cost for the entire project (or release). Our job is to discover and explore what will work best. It’s implied that in this “Age of Accelerations” and very volatile requirements, we are under much more pressure to deliver quickly, and there are tight schedule constraints. We need the team of 5 to 10 senior, colocated, “generalizing specialists” to help us meet these tight schedule constraints.

So, what is the best way to handle these types of situations? For software applications or “knowledge work” projects based on intellectual work and design work, the best approach is to do things iteratively; use Agile. In very short iterations—one to four weeks—get a subset of the application out to the customer, and let them kick the tires so to speak, and find out what has value and what does not. The Agile lifecycle is going to look as given in Figure 1.4.

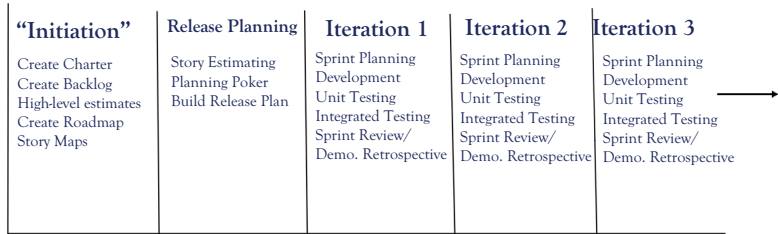


Figure 1.4 Agile lifecycle

Once we get through the initiation phase and the release planning phase, and into the “development iterations,” each development iteration is a vertical slice of the entire project, and aspects of all phases are used in each development iteration. (Also, it goes without saying that all five process groups are used in each iteration.) In each development iteration, we will do planning, we will execute against those plans to create “product increment”—something empirical and tangible (usually a prototype of some deliverable, but not a throwaway prototype) and something that can go into production. In each development iteration, we will also do testing and quality control (QC) and get acceptance from the product owner and/or customer.

A key aspect of what happens using Agile approaches and doing things iteratively is that we are using Lean. Lean is a core part of all the different Agile methodologies. With Lean, we are “grooming the value chain” and “identifying fast failures” (or identifying failures fast). When we go to obtain requirements from our customers, what are they likely to give us? Everything! Really! Everything under the sun remotely imagined for this project. And ... everything is priority one! Doesn’t the federal government do this in most of their requests for proposals (RFPs)? As I remember things, for a large federal IT procurement RFP in the 1990s—TAC4—there was so much in the RFP that even a huge company like mine (Hewlett-Packard) could not meet all the requirements on our own. We had to find vendors and subcontractors, who could fill the gaps of what we could not provide to have any hope of winning this opportunity. Was the federal government likely to use all these requirements? Never!

In one Standish survey of several years ago, it was quoted that “65% of the requirements the customer thinks they absolutely have to have will

never be used.” There is some controversy about how the Standish group came up with this 65 percent number, but another way of viewing the situation that is widely accepted is another version of Pareto’s law. For Quality Management, all my PMP® Prep students know that Pareto’s law is “80 percent of the problems come from 20 percent of the causes.” In sales, Pareto’s law is “80 percent of your orders come from 20 percent of your customers.” In requirements, Pareto’s law is “80 percent of the customer’s need comes from 20 percent of the requirements.” Therefore, using Agile, we will go after this 20 percent of the requirements in our first iterations. Very quickly we will get something out to the customer that is empirical and tangible, and demonstrates this “product increment” to them. We will find out what has value and what does not. If features we’ve included in the product turn out not to have value, then we will delete these from the “backlog.” We won’t invest in them further; we won’t have to do further integrated testing, we won’t have to do as much documentation, and therefore, we will save time and we will save money. We will keep “grooming the value chain” using Lean.

Creating something that’s empirical and tangible, and then demonstrating this to the product owner and the customer, also has tremendous value. It is far more valuable to show them something empirical, than just have them review large documents of specifications. In a wonderful quote from Doug DeCarlo in his excellent book, *Extreme Project Management*, he says, “If a picture is worth a thousand words, a prototype is worth a thousand pictures.” The product increment we are creating in each iteration is very similar to a prototype, but it is not just a model, or a mockup of the deliverable that will be thrown away. What we are creating is a subset of the overall application or solution, but it is something that will go into production.

In *Scrum: The Art of Doing Twice the Work in Half the Time*,⁸ Sutherland gives us a number of real-world examples of the benefits of Agile. One of these case examples was from 2006 and involved Medco.⁹ At that time Medco was the world’s largest online pharmacy.

⁸ Jeff Sutherland is one of the coauthors of *Scrum* along with Ken Schwaber.

⁹ *Scrum: The Art of Doing Twice the Work in Half the Time* by Jeff Sutherland, J.J. Sutherland p. 111.

Here are some key bullet points from this case example:

Medco, 2006

- At the time, Medco was a Fortune 100 Firm: world's largest online pharmacy.
- Medco planned to rollout a new online pharmacy that would link patients with a pharmacist who would review all their different prescriptions and ensure there were no conflicts. Furthermore, they would guarantee a significant cost savings for patients moving to this online pharmacy.
- The CEO had checked with the IT management team and received assurances this was technically feasible, but he did not get any details on how long it would take!
- He promised Wall Street that this new online pharmacy capability would be available in one year, by July 7, 2007!
- Wall Street loved this entire idea, and Medco's stock took a very nice jump upwards.
- However, the people who had to implement this capability did not learn they would be going forward with the massive project until after the CEO had made the commitment to Wall Street! But the challenges to implement this were immense!
- Much of the software the company relied upon to direct on-site robots was badly outdated.
- In Medco's five gargantuan plants, filled with 4,000 pharmacists processing prescriptions, robots whizzed about pulling pills while other robots handled packaging and mailing, and all those systems had to talk to one another with 100 percent accuracy, or someone would die!
- It took the company six months to figure out they couldn't do it on time.
- In the *best-case* scenario, it would be a year late!
- They thought they could plan everything ahead of time. They spent months of effort making a detailed project management plan with pretty charts. This was all fiction!

- There were reams of paper outlining requirements, compliance, all sorts of reports, and quality assurance (QA). The stack of requirements documentation was two feet tall.
- There were so many people involved and at loggerheads that everything was deadlocked.
- So, they used the Scrum process to pull out the most essential requirements (backlog items).
- They wrote all these items down on sticky notes, plastering the walls with hundreds and hundreds of sticky notes. (This process of pulling out the minimum viable product items and writing them down on sticky notes took several hours.)
- Then, they conducted a high-level estimation of the backlog items.
- And, finally, “What do we need to do first?” Prioritize the work—what is the “20 percent of the requirements that would fulfill 80 percent of the need?”
- Bottom line—Medco met their July 2007 date!

I believe many Agilists miss the key point of using Lean. They think the key purpose of using Lean with Agile methodologies is eliminating and reducing waste. Here are the classic seven key types of waste (from Poppendieck) that we focus on eliminating:

- Partially done work
- Extra processes
- Extra features
- Task switching
- Waiting
- Motion
- Defects

Eliminating this type of waste is important, but this is not the key thing we are achieving using Lean. As we are seeing in this Medco example, the most important benefit of using Lean for our projects, today, is identifying the 20 percent of the requirements that will provide 80

percent of the customer's need and continuously grooming the value chain. We are going to save time and money if we do this right. Involving the customer on a much more frequent basis to review what the team is creating is also instrumental to achieving this goal. If we use Lean properly, we are going to create something of higher value as we iteratively move through the project than if we used a predictive planning approach to figure out everything, in detail, at the beginning. Starting out, we can't predict exactly what it is we will achieve at the end of the project, or at the end of the release, but if we use Lean properly, it will be of higher value!

At a College of Performance Management (CPM) meeting I attended a couple of years ago (this was a meeting attended by more than 50 senior EVM program managers), the attendees were trying to solve the problem of how they could use Agile effectively with their EVM programs. The federal government was demanding that they do this, and frankly, the program managers were scratching their heads and wondering how and why they should do this. They could see the benefits of running daily standup meetings in many of their projects, and using burndown charts and Kanban Boards, but from my vantage point, they were missing the whole point of why we use Agile. It's really about reaping the benefits of using Lean, especially for software and "knowledge work" projects. As I'll discuss further, if we do this right, we can achieve a 300 percent or greater improvement in our productivity! After the meeting, in the "social networking part" of the meeting, I tried to explain this to one of the EVM program managers, and he looked at me incredulously and said something to the effect that I was an Agile zealot. (I'll leave out his exact phrasing!) I replied back, "No, I'm really not. I'm an enthusiastic hybrid zealot!" (Again, my exact words will be omitted!)¹⁰

In Agile, we're actually improving and speeding up the classic plan-do-check-act (PDCA) loop. The PDCA loop was given to us by Walter Shewhart and W. Edwards Deming. (Shewhart originated this idea in 1939, and Deming added on more thought for its use in 1950.) The PDCA

¹⁰ For another example of how Agile allows to quickly identify the 20 percent of the requirements that will meet 80 percent of the customer's need in the first iterations of a project, see the section in Agile Contracts on "Money for Nothing and Change for Free" contracts.

loop has become the foundation for the best accepted practices in Quality Management for how to achieve continuous improvement, or Kaizen, on our projects. (“Kaizen” is the Japanese term for continuous improvement. “Kai” is to alter or to change; “Zen” is to improve.) The idea is to “Make small incremental, continuous improvements in all aspects of your project; your products, your plans and documents, and even your processes. Don’t try to solve everything at once; don’t try to ‘boil the ocean’; do things incrementally and iteratively.” Of course, this shouldn’t only be applied to projects; it applies to both projects and operations. It applies to all steps in the entire product lifecycle. In fact, the PDCA loop and Lean and the Toyota Production System have their origin in manufacturing processes and operational processes. The best of the modern proprietary quality methodologies—Six Sigma, Toyota Production System—Lean, Capability Maturity Model Integration (CMMI), Just-in-Time (JIT), ISO-9000 (International Organization of Standardization)—are focused on improving operational and manufacturing processes, and using the PDCA loop is emphasized in all these proprietary methodologies.

Lean later evolved into the Lean Software Development movement as a subculture of the Agile movement in the early 2000s. But, interestingly, in the *PMBOK® Guide*, for determining the business case for the project, and in the Cost Management chapter, doing “lifecycle costing” has always been emphasized. Thus, this is a part of traditional project management. The idea is, as you do your project, don’t take a short-range view. Don’t make decisions that save money and costs for the project lifecycle where such decisions could lower quality on the product or make it hard to maintain the product in operations and, therefore, hurt the company in the long term. Take the long-term view. Design the product for the long run, so it is easy and practical to maintain and has the right level of quality (so it doesn’t have to be repaired frequently!) to protect the company into the future. Therefore, as we do our projects, protect the entire product lifecycle. Design in the right QC processes, make sure the project is consistent with the company’s quality policies and methodologies, and that process improvement processes have been designed into the project. Use the best-quality processes inside the project itself.

In Agile, we’re speeding up this PDCA loop in some very important ways. On a classic software development project (circa 1978!), how often

did we meet with the customer? Not very often! We met with them at the beginning of the project to obtain user requirements and to write the functional specification, and then, we went away for almost two years to develop the application. It wasn't until UAT—often times two years after the start—when the customer saw the real, running application. We might have demonstrated some mock transactions or models of the application at various times to them in the interim, but it wasn't the real application with real response times, throughput times, and actual transactions. So, again, there was usually some real disappointment at UAT with the application we had developed. In Agile, we're taking a very different approach. We're speeding up the feedback cycles with the customer in important ways, but also between all members of the project team.

The feedback cycles being used on an XP project are given in Figure 1.5.

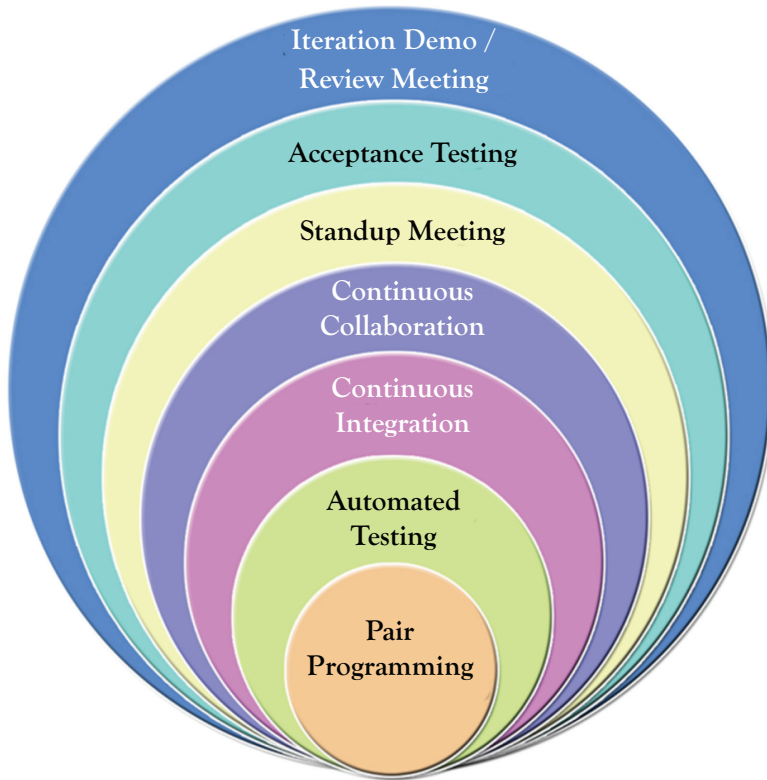


Figure 1.5 Levels of feedback and improvement and knowledge sharing—XP + Scrum

- Using “pair programming”: code is tested in seconds.
- Unit test/automated testing: There is feedback in minutes.
- Continuous integration: Feedback in hours.
- Customer collaboration: Feedback in hours.
- Standup meeting: Feedback once a day.
- Acceptance test: Feedback in days.
- Iteration demo (review meeting): Feedback in weeks.
- Release review meeting: Feedback in months.

Increasing the frequency of communications with the customer and the feedback loops within the entire team significantly reduces the cost of change on a project. When I am teaching a PMP® Prep class, I get to a juncture in the class where I ask, “When do stakeholders have the most influence on a project?” And the students readily reply, “They will have the most influence, of course, at the very beginning of the project.” And the reason is simple: the cost of change is the lowest at the beginning of the project. Once blueprints have been created for the project, then the stakeholders’ influence has decreased, because if they change their mind on the design, they will have to throw out the existing blueprints and get new blueprints created. Of course, that is expensive. Once the actual construction of the building begins and a key stakeholder changes the design for the building, then the cost of change is prohibitively high. Their influence to change the project at that point is very limited; their influence decreases as the cost of change increases. So, we show the students a slide where we see the cost of change as a line running diagonally to the influence of stakeholders. It would look like that in Figure 1.6.

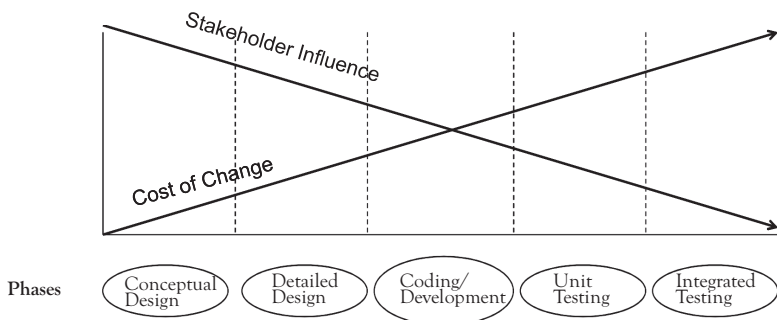


Figure 1.6 Stakeholder influence and cost of change in traditional projects

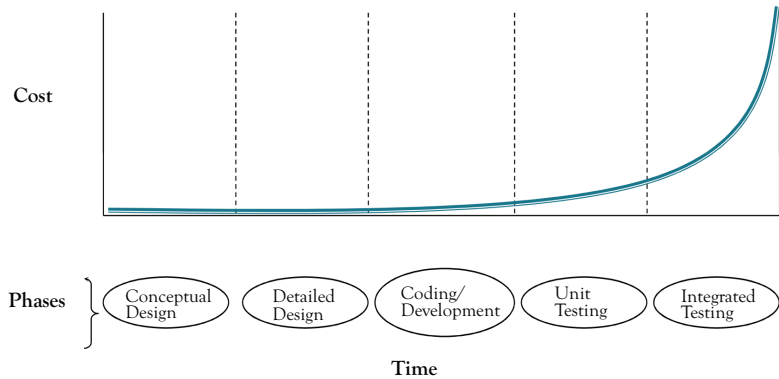


Figure 1.7 *Real cost of change in traditional projects*

However, for a project where we are creating a new software application or for a “knowledge work project,” the cost of change line is not really a diagonal line. Since there is a lengthy period of time between the requirements gathering process and UAT—oftentimes more than a year—the cost of change line will have a nasty curve or uptick at the end of the project. Every change the customer wants at that point will have tremendous ramifications. Every change will touch many other parts of the project and will be very expensive. The cost of change line ends up looking like that in Figure 1.7.

With Agile approaches, by increasing the frequency of communications with the customer, by increasing the number and frequency of feedback loops, and by speeding up the PDCA loop, we flatten out the cost of change curve. Since we are meeting with the customer at minimum every month in iteration review meetings—but, in most cases much more frequently than that—the cost of change line looks like as in Figure 1.8.

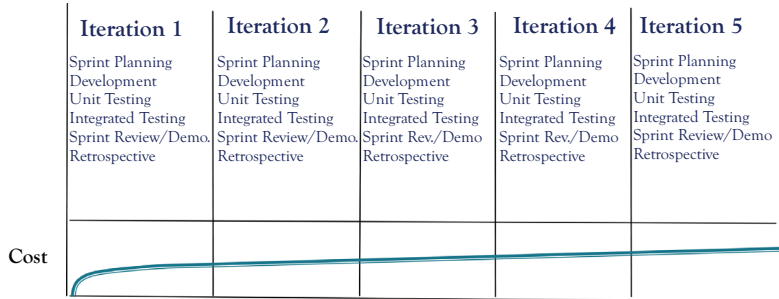


Figure 1.8 *Reducing the “cost of change” with Agile*

Jeff Sutherland gives us a couple of examples of how speeding up the feedback cycle and also the PDCA loop can reduce the cost on a project. These examples are provided in the section entitled “Do It Right the First Time” and “Fix Problems as Soon as You Find Them.”¹¹ The first example involves a lesson learned at Palm in the early 2000s with their production of applications for personal digital assistants (PDAs). These devices were the predecessors of our smartphones today, and originally, they provided a number of applications such as a calendar function, spreadsheet software, and a notebook function. Palm learned that if a bug was discovered in a new application being developed, if the programmer fixed the bug immediately, it took one hour to make the fix. If the programmer waited three weeks, it took 24 hours to make the fix, or 24 times longer! A second example contrasts the way Toyota produces a Lexus today, with the way a Mercedes or BMW is built. A Lexus is made in 16.8 hours with 34 defects per 100 cars. If anyone on the team discovers a defect, they can stop production until the defect is fixed. In contrast, a BMW or Mercedes is made in 57 hours, and the German manufacturers do their quality testing after the car is produced. They spend more time fixing problems than Toyota does in producing their car!

Using Agile and Lean can provide tremendous advantages to us and help us avoid some of the classic pitfalls with our #1 Risk: a poorly written scope statement, or not being on the same page with the customer. This is especially important in our modern world of very volatile requirements and tremendous change: Tom Friedman’s “world of accelerations.” Also, providing something empirical and tangible to the product owner (and customer) on a very frequent basis has tremendous advantages. Again, Doug DeCarlo’s quote is especially apt here, “If a picture is worth a thousand words, a prototype is worth a thousand pictures.”

Sutherland says that the normal productivity gain from using Scrum will be 300 to 400 percent! Some teams have achieved an 800 percent improvement in productivity. He cites another case where a business friend was very impressed with a 25 to 35 percent improvement that they had achieved using Scrum. His immediate reaction was, “You must

¹¹ *Scrum: The Art of Doing Twice the Work in Half the Time* by Jeff Sutherland, J.J. Sutherland p. 97.

be doing something wrong!” The key benefit is not providing new tools for the development team. No, the key benefit is much more frequently demonstrating product increment to customers—the product owner—and determining what has value.

As we said, Lean is actually something that needs to be used all across the entire product lifecycle: from business development, through the project lifecycle, through testing, through QC and QA through integrating in Cybersecurity, and into the full part of the operational lifecycle. This is a key point the DevOps authors are making. In the *DevOps Handbook*, the authors say that after the manufacturing revolution of the 1980s, driven by Lean principles and practices and the Toyota Production System, “Organizations that did not implement lean practices lost market share, and many went out of business entirely.”¹²

So, Agile and Lean are the solutions to all our problems, yes? (Silly rhetorical question, of course!) No, there is no one perfect process or solution to solving all our project management problems! There is no one process that’s a panacea or which provides an easy way out. We have to be more flexible than that and realize we may have to use multiple processes and approaches. We’re going to have to mix and match.

More than that, project management is hard work. That is just flat-out the case. Getting everyone onto the same page and prioritizing and getting agreement on requirements are hard work. We’re negotiating, facilitating, communicating, and using all the soft-skill parts of project management to reach that goal. We’re often working with very powerful stakeholders, who outrank us in our own organization or in outside organizations, and these stakeholders all have different needs, different wants, and different expectations for the project. This isn’t going to be easy! There is a lot of pressure to meet very tight schedule constraints and budget constraints and achieve high quality. No pressure! The emphasis is clearly on the EQ part of the equation, not the IQ part of the equation.

Agile—by increasing the frequency of communications and by using “low tech and high touch” communications—is helping in key ways. But again, it’s not a panacea.

¹² *The DevOps Handbook: How to Create World-Class Agility, Reliability and Security in Technology Organizations*—Gene Kim, Jez Humble, Patrick Debois, John Willis and John Allspaw—Location 515.

In the classic PMI® model (defined in the *PMBOK® Guide*) the PM is accountable for the project and for making all this happen. In the traditional or classic scenario, management wants that single point of contact for the project, or that “one throat to choke!” In the Agile model (especially with Scrum and XP) the closest role to the PM is the *Scrum Master* or the *Coach*. In Agile, the *Scrum Master* is not a “large and in-charge PM” or a traditional, fully accountable PM. The Scrum Master is a “servant leader” and his or her job is to make life easier for the team members.

Scrum Masters make sure other team members aren’t interrupted and interfered with during the Sprint/Iteration. So, their job is “to carry food and carry water!” They are the owners of the methodology and the go-to person when there are questions about how to best use the methodology. However, the “team” all together (the developers, the Scrum Master, and the product owner) is accountable for the project! How can that possibly work? How can multiple people be accountable? Management is surely going to want that single point of contact. Yes? (That “one throat to choke!”) Actually, this can work, and does work, because of the frequency of communications and the frequent feedback cycles. The idea is that management (and also the product owner) can wait one to four weeks for the end of the Sprint/Iteration for the review meeting to inspect the product increment (deliverable that was created in the Sprint), decide if this deliverable(s) meets the “definition of done,” and decide whether it is acceptable. The entire team is accountable for what has been created in the iteration and what is being demonstrated in the review meeting.

Problem Areas for Agile

Agile is not a perfect solution for all projects. What are some key problem areas and some key types of projects where Agile is not a good fit, and why? Here are a few examples to use as a starting point for our next section on “How Do We Make Hybrid Approaches Work?”

1. If we do not have the right type of working relationship with our key stakeholders—the sponsor and other senior managers, customer, functional managers, and other stakeholders—then Agile is not going to work as it should. As was said before, it is best to use Agile for projects when the customer doesn’t know exactly what they

want starting out in the project, and we need to discover and explore requirements. Thus, there is a premium being put on creativity. Therefore, the team needs to be provided a lot of freedom and trust in exploring what the best solution might be. It follows that Agile is not going to be the best fit in the following circumstances:

- If there is a lack of trust or working experience with all the stakeholders and stakeholder groups.
- If the customer and management do not accept that we are in a “time and materials world” where we need to discover and explore requirements, then Agile is not going to be the best fit. Management must understand the benefits of Lean and Agile and must understand they need to be involved in the review meetings and the frequent feedback loops for this to work. If they are insisting on fixed schedule constraints where fixed detailed design objectives must be met, this is not the right fit.¹³
- If management thinks Agile is just a nice set of tools for the developers and programmers that will increase their produc-

¹³ The acceptance criteria, or “definition of done,” will be more high level (as in a Dynamic Systems Development Method, DSDM, contract type) than they would be in a fixed-price (FFP) contract. They will be about high-level functional requirements, not detailed engineering design requirements. A number of different engineering designs could meet the functional, high-level requirements, and we are entrusting the team to explore and discover the right engineering design. So, management must be accepting of this. There must be more flexibility, trust, and tolerance in assessing what meets the definition of done. But we are more in the world of IKIWISI—“I’ll know it when I see it!” So, management must understand this well. We can’t start out the project with preconceived notions and demands for the final detailed engineering design. Also, we can’t have the situation where we sell the concept of Agile and Lean to management; at the outset of the project they profess to be quite accepting of this approach, and say they see the benefits; then, in the middle of the project, “life happens!” Some emergency comes up, and management quickly reverts back to a classic mindset and insists on fixed constraints for time and cost, but also fixed scope requirements that are detailed engineering requirements. This was not the agreement between management and the team starting out. We must have a strong relationship between management and team and know there won’t be such a change in mindset part-way into the project.

tivity and they do not have to get involved, then this is not going to work. Using Agile will not meet expectations.

- If management doesn't provide the right culture for the Agile team members, a "Theory Y type environment,"¹⁴ empowering the team and providing a lot of freedom and trust, then this is not going to work well.¹⁵ We will explore this topic in more depth in the section "How Do We Make Hybrid Approaches Work?"

¹⁴ "Theory X" companies and "Theory X" managers start with the belief that their employees do not enjoy their job and do not enjoy work. If you start with that premise, then you will believe your employees will be lazy, careless, inefficient, and unmotivated. You will have to micromanage them closely to achieve the right business results. "Theory Y" companies and managers start with the opposite belief. They believe their employees do enjoy work, want to do a good job, and if they are provided freedom and trust, they will achieve the right business results.

¹⁵ Many Agilists think they invented this enlightened culture of providing freedom and trust to the team members (the "Agile Ethos"). They did not! I believe this culture really started with Hewlett-Packard in 1939 and was a core part of the "HP Way." This became the foundation of most companies in Silicon Valley from the mid-1970s through the early 1990s. Walter Isaacson, in his book, *The Innovators: How a Group of Hackers, Geniuses, and Geeks Created the Digital Revolution*, describes very well how this culture originated in HP in the early 1940s, and later spread through other Tech companies such as Intel in Silicon Valley. Bill Hewlett, in defining the HP Way, said, "We believe people want to do a good job, and we're going to provide them that opportunity. We are going to provide them the right environment, but then trust that they will achieve the right business goals." ... "We believe the people closest to the problem will have the best solution. We will not micromanage them. We will define the high-level strategic goals, but empower people closest to the problem to come up with the right solution." ... "We also believe that people are our most valuable resource." ... "If we succeed as a company, our people should succeed." They backed this up with a very enlightened profit-sharing program and employee stock program at a time when this was almost unheard of. It breaks my heart that in the early 2000s with new management, very large mergers and acquisitions, much of this amazing culture began to be dismantled. Other Silicon Valley companies also seemed to depart from this enlightened model, but the most successful Tech companies of the past 10 to 15 years, Google and Apple, have kept this culture alive and well. I think keeping this culture alive was a big part of their success!

- If we are subcontracting much of the project work to vendors, and we have very little past-performance information for these vendors, then Agile is not a good fit. We will need a detailed SOW with all the I's dotted and T's crossed.
2. Large, complex projects with many stakeholders and stakeholder groups—difficulty finding the right product owner who can resolve conflicts.
- Jeff Sutherland describes several projects¹⁶ (the FBI Sentinel Project, Medco, etc.), which were all in crisis and failing, but in all these projects there were a large number of stakeholders and stakeholder groups that had disparate needs and wants that were not compatible, and the projects were stuck in a deadlocked position. The stakeholders were at loggerheads about what should be included in the project and what the priorities should be. One of the first goals in transitioning to Scrum is to simplify this type of situation. For the FBI Sentinel project, they reduced the staff from 220 to 40 and reduced the number of FBI employees on the project from 30 to 12. The next step focused on prioritizing the requirements—or the product backlog—and identifying the 20 percent of the requirements that would achieve 80 percent of the need. Having the right product owner, if that can be achieved, will definitely help resolve deadlocks in choosing what should be included in the product backlog and the priorities of these backlog items. But, on some very large complex projects, it may be necessary to have multiple product owners and, of course, that can present problems too. Someone has to have the final say and be the decision maker. And, if the right product owner is not selected, and if senior management ends up vetoing what the product owner had previously approved, but only after three or more months have passed, then we will have wasted a tremendous amount

¹⁶ *Scrum: The Art of Doing Twice the Work in Half the Time* by Jeff Sutherland, J.J. Sutherland.

of time and a lot of money, and Agile didn't work the way it was intended.

- So, it might be difficult to find the right product owner (or “Customer” in XP). A key assumption for Scrum (or for XP) is that we can find the right product owner, who will effectively resolve the conflicts between incompatible requirements for what will be included in the project and define priorities. But, on a large, complex project with many powerful stakeholders, it may be very difficult to get agreement and get all the stakeholders to buy into what the product owner has decided. Again, as we described earlier, this is the hard part of project management! Whether it is a PM or a product owner striving to get agreement on inconsistent, incompatible requirements, this can be a very tough challenge to handle. Many large, complex projects and programs (F-22, Affordable Care Act website, etc.) have failed because this wasn't handled correctly. No matter what powers have been given to the product owner, there will be many other stakeholders on the project (e.g., the sponsor, the customer, other senior managers, federal regulatory agencies) that outrank the product owner. The skillsets the product owner must employ to get everyone onto the same page are “soft skills”: negotiation skills, communication skills, facilitation skills, and so on. Following a particular process, or even a methodology, will not guarantee success in achieving this goal. This is about the “EQ” part of the equation, not the “IQ” part!
3. We are in a “cookie-cutter” type project and have successfully done very similar projects like this one many times before. There were only small customizations differentiating the projects. (Again, does that mean this type of project is easy? No!) However, if we are in a “cookie-cutter” type project, it will be far less expensive and more efficient and practical to use the traditional approach.
 - We don't need to discover and explore requirements: The customer knows going in exactly what they want for this project. We need a traditional SOW where all the “I's are dotted, and T's are crossed!” (e.g., A young couple is remod-

eling their kitchen, and the key stakeholder—the wife, of course!—knows exactly what she wants going in—the make and models for the different appliances, the exact type of cabinets and countertops, etc.)

4. We are handling an emergency situation, and there isn't time to explore or discover requirements: There isn't time to prototype. (There might not be time to canvass all the stakeholders and exhaustively detail all the requirements either!) We may be in the type of situation where "A less than optimal decision made now—at the right time—is better than the perfect decision made too late."
5. We have only one chance to do this project, and we have to get things right the first time (e.g., again, perhaps an emergency rescue mission: Apollo 13!).
6. Places where more central control is needed. (The classic PMP*—fully accountable PM is needed.) Perhaps this could be for an emergency situation or a military operation. There isn't time to debate different solutions; we need the team members to follow through on the plans being disseminated by the PM.
7. If the team members are multitasking on a number of projects at the same time, are only able to perform very specialized tasks within their range of expertise, and are only working part-time on the project, then Agile probably isn't the best fit.
8. If the team cannot come up with an "MVP" (minimum viable product) to demonstrate to the product owner and customer in the early iterations, Agile is probably not a good fit. For a large, complex project, it might take a development team many months to tie together all of the infrastructure and hardware before any kind of recognizable demo or MVP can be made. In the early iterations, it might only be possible to demonstrate something using prototypes or models with "fake data." Engineers and customers could sit side-by-side with the product owner and customer, discuss the prototype, and talk abstractly about what features the customer would like the real product to have or how they would like to use it. But then there will often be a large effort and a long period of time to get a real working product created that is usable by customers. These "planning phases" of the project could take months, and a predictive

Agile project management and stage-gate model—A hybrid framework for technology-based companies

By Edivandro C. Conforto, Daniel C. Amaral

Section 3. The hybrid framework: iterative and visual project management method

One solution might be the integration of the APM approach with stage-gate process and practices. This focus is considered a relevant research agenda, mainly because APM is still seen as a management approach designed for software development companies (see Conforto et al., 2014), but with great potential to be customized for other industries and projects. Furthermore, recent studies have shown the importance of adapting and combining APM practices with more traditional NPD processes (e.g., stage-gates) to improve flexibility and response to changes in uncertain and dynamic project environments (Cooper, 2008; Högman and Johannesson, 2013); in addition, recent studies have indicated the potential impact of agile practices in project performance, as discussed by Serrador and Pinto (2015, p. 1049) based on a survey with 1002 respondents from different industries.

APM focuses on dealing with constant changes by enhancing interactions with customers, developing people's specific competencies, and planning and controlling projects through short iterative cycles so as to cope with the challenges of dynamic project environments in the software development industry (Highsmith, 2004; Schwaber, 2004; Augustine, 2005, among others).

Agile methods are focused on flexibility by using a minimal set of rules and eliminating activities that do not add value to the product development process. They are based on series of iterative development cycles and promote self-management and self-discipline attitudes in order to help the team be more responsive to changes (Beck, 1999; Boehm and Turner, 2004; Cockburn, 2004; Cohn, 2005; Highsmith, 2004; Schwaber, 2004).

Discussion about integrating agile practices with traditional NPD processes is not new. According to Cooper (2008), managers should consider different approaches, such as "iterative development", to deal with different types of projects. This evidence was confirmed in a recent study by Högman and Johannesson (2013) in six hardware development companies. From this perspective, APM combined with the stage-gate approach could be explored as an alternative to deal with the challenges of managing product development projects in highly dynamic business environments.

Conforto and Amaral (2010) proposed a hybrid framework that combines APM with stage-gate process, the IVP2. The framework combines stage-gate model elements, such as phase definition, standardized documents and deliverables, milestones and phase checkpoint reviews with iterative development (e.g., 2-week development cycles or "sprints"), multiple levels of planning and controlling, visual boards (e.g., Kanban), and frequent informal meetings. The IVP2 is an example of how to implement Cooper's (2008) insights by combining iterative development with stage gates, and it is aligned with the demand for hybrid models to meet the needs of some specific types of organizations (Högman and Johannesson, 2013; Marion et al., 2012).

3. The hybrid framework: iterative and visual project management method

This section presents a synthesis of the hybrid framework, IVP2 (Conforto and Amaral, 2010), which was adapted to the project under analysis. The sole purpose of this study is to present empirical data regarding the use and effect of the framework in the project and product process and overall performance.

The IVP2 combines different concepts from the NPD and APM literature (e.g., Cooper, 2001, 2008; Boehm and Turner, 2004; Highsmith, 2004; Schwaber, 2004; Augustine, 2005; Cohn, 2005; Shenhar and Dvir, 2007). The framework (IVP2) implemented in this project comprises five dimensions and a seven-stage iterative cycle. The five dimensions are: (1) phase and project deliverable model (PPDM); (2) project planning and controlling whiteboard (PPCW); (3) weekly activity planning whiteboard (WAPW); (4) project management software (PMS); and (5) simplified performance indicator system (SPIS). The seven-stage cycle represents an iterative development approach throughout a phase-oriented development process (stage-gate); each project phase can have as many iterations as necessary in order to satisfy the requirements for each phase-review milestone (or phase gate) of the product development process.

Fig. 1 illustrates the IVP2. One of the key concepts used in the IVP2 is multiple planning and controlling levels, which are supported by some authors in the APM literature (Boehm and Turner, 2004; Cohn, 2005). IVP2 has three levels. The first level is the stage-gate model represented by the PPDM, its phases, milestones, and macro-deliverables for the entire project. These deliverables are related to the product and technology development phases.

The second level is related to iterative development, also named "sprints" in the Scrum agile method (Schwaber, 2004), in which the team breaks down the deliverables from the product backlog into tasks or small deliverables, and prioritizes and defines the order of development during the iterative planning meeting (or sprint planning). These tasks are performed in time-boxed iterations (on average a 15-day length). The PPCW represents the general set of deliverables and tasks to be performed in each phase. It is a physical visual board that evolves throughout the project life cycle and could be compared with the product backlog described in the Scrum agile method (Schwaber, 2004).

The third level is on a weekly or daily basis. It is similar to the iteration plan or sprint backlog (Schwaber, 2004). The WAPW is related to the tasks to be performed on a weekly or daily basis during the iteration. The IVP2 deliberately recommends these three levels. The information generated on all three levels can be registered in the PMS.

Once a new project begins, the iteration cycle starts with the definition of the main phases and project deliverables, guided by the PPDM (Stage 1). This element of the IVP2 is important because its purpose is to guide the team into the main phases of the product development process. Therefore, the PPDM includes all main deliverables and documents which are key outputs related to the product and project being executed, and these elements are specific to the type of project and industry sector.

Once the PPDM is defined and the team has a broader view of the macro-deliverables for the project, the project manager and team members collaboratively define the iterations and place the deliverables related to the product on a visual

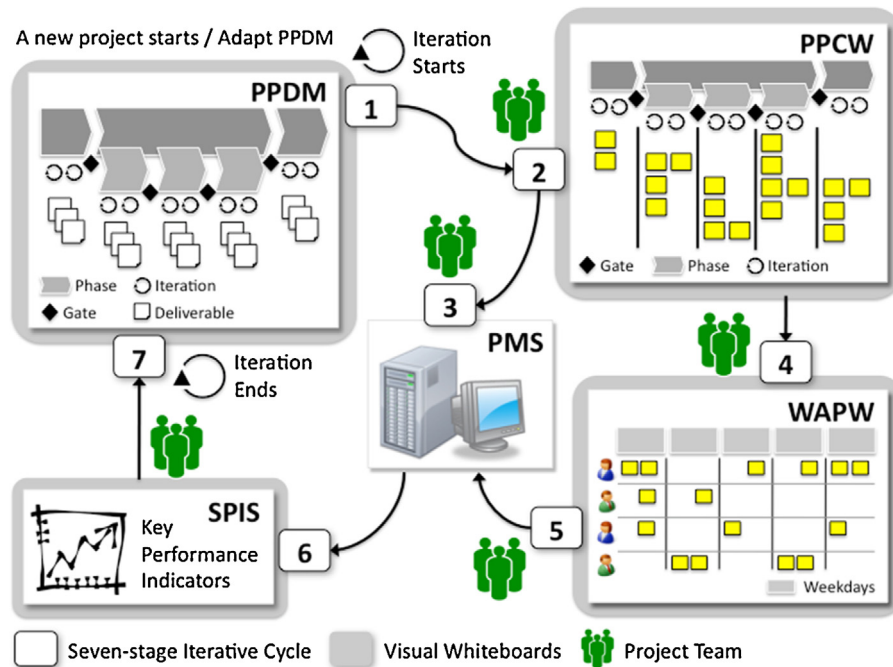


Fig. 1. IVPM2 combining APM and NPD process. Source: Conforto and Amaral (2010).

whiteboard (PPCW) using sticky notes (Stage 2). At this stage, the team defines the iteration length (e.g., usually 2 weeks, 15 days). However, this length may be dependent on the type of project. According to the IVPM2 framework, the phase length is usually longer compared to the iteration length, and it reflects the characteristics of the project type and industry sector.

At Stage 3, the deliverables placed on the PPCW (product backlog) are registered in the project management software. In the project studied, the team used an open-source software that has the most common features of commercial project management software. The PMS is used in the IVPM2 to document and register project data and information regarding cost, schedule, and progress to help the team generate progress reports to be used in the evaluation of the phase review meeting as well as the iteration performance. The PMS also helps the team create different views of planning and scheduling, for example, the Gantt chart, along with the visual board used to illustrate the iterative development cycles, the WAPW and PPCW.

After being fed into the PMS (Stage 3), the deliverables must be detailed into activities and tasks to be executed during the iteration (or sprint) and placed in the WAPW (Stage 4) to illustrate the work to be performed on a weekly basis. Project team members perform the tasks related to one or more PPCW-defined deliverables, and once these tasks are complete, members need to update the information and project plan in the PMS as well as the visual whiteboards (Stage 5).

At Stage 5, the development is on a weekly and sometimes daily basis, supported by rapid and focused team meetings called “stand-ups” or “daily Scrum” (Schwaber, 2004). In other words, the team regularly discusses what members need to deliver, and what the issues or challenges are, so the project manager can proactively address these problems. At Stage 6, using the PMS, the project manager generates performance reports and provides an overview of the project progress to the team. At the end of the iteration, the project team and project manager use the information to make decisions, to improve the process, and to discuss upcoming risks or obstacles in order to plan for the next iteration cycle (Stage 7).

The IVPM2 relies on an iterative planning–exploring–delivering approach. It combines key elements from the stage-gate model and APM literature, as recognized by Gonzalez (2014, p. 9). The primary goal of having a phase-oriented model combined with iterative development is to allow multiple levels of planning and execution in a hybrid management approach, allowing a balance of discipline and flexibility for highly dynamic and innovative project environments, such as the case reported in this paper.

4. Research approach and methods

4.1. Case description

This study is based on a case study approach (Eisenhardt, 1989; Yin, 1993; Voss et al., 2002) and primarily focuses on collecting perceptions about the usefulness and contributions of the IVPM2 framework for the project and product development process performance as well as its adherence to some of the key APM principles found in the literature.

Thriving Teams

By Werner Zorman

Core Theory,	pages 11-14
Connecting Across Universes,	pages 23-24
Feedback,	pages 58-59
Team Culture,	page 98
Intervening with Intention,	page 124

core theory (c3)

I don't know about you but I have a hard time remembering words. I often use memory hooks and I am glad that all three elements of my core theory start with a "C" which may help those of you who also struggle with remembering.

The first 'c' in the core theory stands for **constructivism (con)**. Depending on your perspective, constructivism can be seen as a belief that our differences separate us greatly and that we can never truly know what's out there; or for the optimists among you, as a recognition of connection and the miracle of our universe. Either way, constructivism asserts that every human being, including you and me, experiences their own version of the same universe that we all inhabit.

The second 'c' stands for **communication (com)**, which is crucial for the existence and survival of social systems, including your student team. I'm not referring to the sending and receiving of information, which may suffice to describe computer interactions. Instead, I use a model of communication that describes it as a series of intentional utterances produced by you and your teammates, with the potential to create a shared understanding of the conscious and unconscious, spoken and unspoken elements.

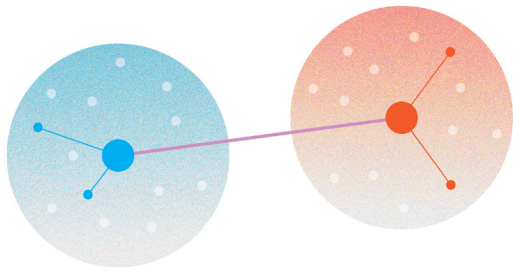
The third 'c' stands for **cybernetics (cyb)**, which has nothing to do with cyborgs or virtual reality. It has everything to do with system thinking. Your student team is a system with its own dynamics, norms, and development and is impacted by the forces of disorder as everything else in our universe. To resist this entropic force, your team must learn how to become a self-organizing system that continuously learns and adapts to its environment. Cybernetics is in my opinion the most underrepresented topic in leadership- and team development theory, which is likely due to the US's obsession with individualism.

In line with my memory hook approach, I have come up with "three word expressions" (3we) for each of the three core elements. Each 3we is starting with a 'c' and aims to describe action-phrases for constructivism, communication, and cybernetics.



Each voice a note played,
Melodies merge, harmony,
Symphony of hearts.

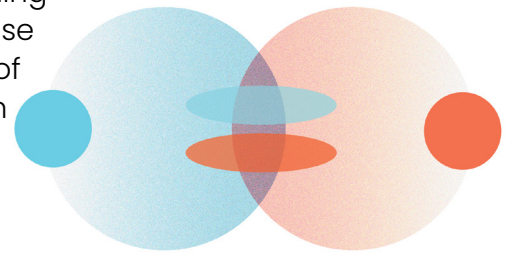
connecting across universes (cau)



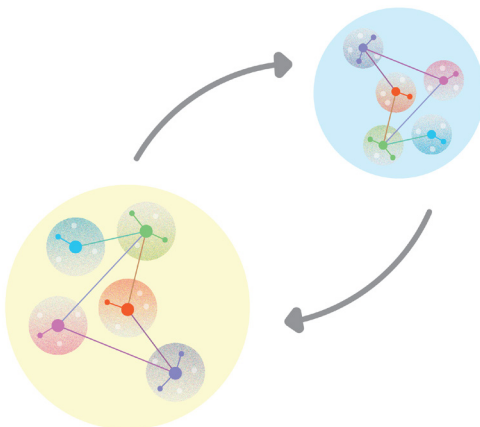
The 3we for constructivism is connecting across universes. When I talk about “connecting across universes,” I’m referring to the process of exploring and understanding the meaning-making of others. It’s about opening portals, connecting with someone else’s universe, and exploring the beauty and shadows with curiosity and empathy. While difficult, I believe it is possible to gain insights into each other’s universes through communicating for understanding. Connecting across universes is not a binary, but a spectrum ranging from ignorance to understanding and from disdain to **love**.

communicating for understanding (cfu)

The 3we for communication is communicating for understanding (cfu) and seems self explanatory. I wrote “seems” because I believe that you likely have a different understanding of the word understanding than I do. The understanding I am talking about requires an ongoing stream of utterances of information and usually doesn’t lead to a fully shared understanding. **Misunderstanding is the norm** when we communicate due to the fact that all of us construct our unique reality. It should be a relief to hear that creating partly shared understanding is sufficient for people to connect across universes and for your team to thrive. Fully understanding is impossible, reserved for the gods, supernaturals, and perhaps the self-aware AI in the future.



continuously self-organizing (cso)



The 3we for cybernetics is continuously self-organizing (cso). Continuously indicates that your team can never stop self-organizing due to entropy being what it unfortunately is. Self-organizing describes first and foremost that your team must observe, assess, adapt, and change itself. **Feedback is the lifeblood** of self-organization and requires honesty, empathy, courage, and practice. Self-organizing also emphasizes the circularity of team members and their interactions establishing the team’s norms which influence these very team members and their interactions. This creates a positive (or negative) cycle where your team becomes stronger (or weaker) and more (or less) adaptable over time which supports your team’s thriving and success (or not). It’s up to you which one it is.



The T-Group and its experiences have also made me a more understanding person. Before, when people who I did not like would come into my life, I would find reasons to not like them and I would generally choose not to get to know them further. Now, I am much more willing to take a step back and recognize that I may have differences with someone, but to look beyond those and give the person a chance.

Back to elaborating on my claim that connecting across universes will help your team more likely to thrive because it supports your capacity to:

- appreciate each other's differences, even celebrate them
- understand why other team members do what they do
- understand each other's strengths and weaknesses
- facilitate each other's learning and change
- connect with each other's humanity
- engage in productive conflicts
- experience joy and even love

You may doubt that connecting across universes can do all this for your team and you are right! Connecting across universes is one of the requirements but won't be enough. You and your team members also must communicate for understanding (cfu) and continuously self-organize (cso) to become a thriving team. For that to happen, every one of you will need to practice also facilitating learning and change (flc).

For now, I am focusing on constructivism and it's time for another exercise which will help you practice connecting with other people's universes. You may, at first, want to do this exercise with one other person, for instance your roommate or your sibling, to gain some practice. After that, you can do this exercise with your teammates, or you may want to skip the practice and just go for it directly. You and your teammates know best.

**E X E R
C I S E**

connecting across universe
(p.g. 190)

How did it go? What did you learn from your teammates' universes? How was the debrief? How did it change your team dynamics? How will you make sure that this wasn't a one time thing? What's your strategy to continue with cau? Was your advisor involved at all?

Remember that whenever your relationship with one or more of your teammates becomes increasingly difficult, you need to stay curious about why their behaviors make sense to them and why you react the way you do. Staying open and curious about somebody else's universe can be challenging and it's worth exploring what can get in the way.



Prompt: give me a list of conditions which hinder people to connect with each other

1. Cultural differences: Cultural differences can create barriers to connecting with others, particularly if there are significant differences in values, beliefs, and behaviors.
2. Social anxiety: Social anxiety or shyness can hinder people from connecting with others, as it may prevent them from initiating or participating in social interactions.
3. Prejudice and discrimination: Prejudice and discrimination can create barriers to connecting with others, particularly if individuals hold negative attitudes or stereotypes about certain groups.
4. Lack of common ground: A lack of common ground or shared interests can make it difficult for people to find common ground and connect with others.
5. Limited social skills: Limited social skills, such as the ability to read social cues or maintain eye contact, can hinder people from connecting with others.
6. Busy lifestyles: Busy lifestyles can make it difficult for people to find time to connect with others, particularly if they are juggling multiple responsibilities or commitments.
7. Negative past experiences: Negative past experiences, such as rejection or betrayal, can make it difficult for people to trust others and form connections.
8. Personality differences: Personality differences can create barriers to connecting with others, particularly if individuals have different communication styles or approaches to social interaction.
9. Misunderstandings: Misunderstandings can occur when there is a lack of clarity or misinterpretation of what is being said, which can lead to confusion and breakdowns in communication.
10. Emotional barriers: Emotional barriers, such as defensiveness, anger, or fear, can hinder effective communication by preventing individuals from listening to others or expressing themselves clearly.

Feedback Dialogue Tips:

- Give feedback in the way the other person prefers it. If the other person strongly prefers candid feedback and you struggle with that, you better start practicing being candid. And vice versa. It's about honoring the other person's preferences.
- Seize the moment and say it when you feel it. You are in awe of your teammates' capacity to include everyone's ideas, tell them right in this moment. Tell them what you saw, what you think about it and how you feel about it. Did they get it? Did they feel how impressed you are by them? Ask them.
- Create shared understanding - always: paraphrase when you receive feedback or ask your teammate to paraphrase your feedback after giving it. Ask questions when you receive feedback or ask them to ask questions when you give feedback. Remember that **misunderstanding is the norm!** Put in the required work to create shared understanding.
- Use SBI without the other person knowing. Your roommate told you that you are difficult to live with. Ask them what makes it difficult. They respond that you don't care. Ask them what you did which makes them believe you don't care. They say it's what you didn't do. And what is it you didn't do? "You never do the dishes". There is your B without having to tell your roommate about SBI. I do this when a student tells me that my class was great or that it sucked by asking them what made it great or why it sucked.
- Never use absolute language like never, always, every time, only, nowhere or everywhere. Examples are "you are always complaining", "you never ask questions", "you only think about yourself", "you are always happy", etc.
- Describe behaviors and not the person. Examples of what not to say are "you are reckless", "you are lazy", "you are a hero", "you are a stand up guy", "you are the best/worst teacher ever". If you are on the receiving end you can use the tip above and ask "Oh, I am sorry to hear that, what makes me the worst teacher you ever had?"
- Encourage constructive feedback. Since most people are hesitant with giving constructive feedback you can help them by sharing your assumptions about a specific negative impact you believe to have on others. "I am concerned about being too long-winded at times and wondering how it impacts others. Have you observed that? I would greatly appreciate your candid feedback." Even if the other person is hesitant at first, you can engage in a dialogue and can help them share if and how they are impacted by your "rambling".

And here is the promised summary:

- Interpersonal feedback is important for your relationships because it helps increase awareness of impact on each other, builds trust, and fosters shared understanding
 - Many people avoid giving timely, candid and caring feedback all together or use scheduled performance reviews or anonymous surveys which are cruel and ineffective
 - Describe the situation, behaviors, and impact (SBI) when exchanging feedback
 - Feedback is supposed to be a dialogue to create a shared understanding
 - Separate feedback from advice giving - talk first about the past and then about the future
 - Understand each others' preferences for receiving feedback and let them guide your way of giving feedback
 - Feedback is often accompanied by strong emotions
 - Start practicing with frequent positive feedback and then constructive feedback
 - Start practicing with content-related feedback and then relationship-related feedback
 - **Only say it when you mean it**
 - No beaming across universes - you don't know for sure what's going on in your teammate's universe
 - Your teammates's opinion (judgment) about your behavior may point towards one of your blindspots
 - Social identities influence how feedback is given and received - stay open, be curious
- The feedback loop is a circular sequence of working, exchanging feedback, identifying gaps, creating action items for change, and back to working



Sincere words we speak,
Empathy guides our discourse,
Growth is our shared aim.

team culture —

Culture eats strategy for breakfast.
—Peter Drucker

What we do is more important than what we say or what we say we believe in.
—Bell Hooks

What you and your teammates do (culture) is more important than what you say you will do (strategy). Here is an example. Your team may have agreed on **listening to each other with respect and curiosity**. That's an agreed on team norm and part of your team's strategy. Your team's culture on the other hand is what actually happens and when, for instance, you **roll your eyes or interrupt your teammate** while they are explaining their idea. This one specific instance isn't your team culture but your team dynamic in a given moment. If the eye-rolling and interrupting becomes a pattern though, your team dynamics will have manifested itself as a team norm. All your lived team norms make up your team's culture.

Team culture: the collective behaviors and attitudes, rooted in the associated values and beliefs, which shape the way team members interact with one another and the way the team functions.

A positive team culture⁸⁴ fosters a sense of belonging, trust, collaboration, inclusivity, and accountability among team members, leading to increased motivation, productivity, joy, and effectiveness.

In contrast, a negative team culture may be characterized by mistrust, lack of accountability, and poor communication, which can and likely will hinder the team's success.

How would you describe your team's culture? What do you love about the way you work together and what would you want to change? At times, your team may be ignorant of specific non-productive aspects of your team's culture. I encourage you and your teammates to be courageous and engage in the following exercise to discover potential blind spots.

**E X E R
C I S E**

team culture blind spots (p.g. 213)

⁸⁴ Check out the "Identifying Opportunities for learning and change" chapter for an example of a student team's positive team culture.

OK, it's time for a summary. When intervening with intention you want to consider that:

- Intervening means that you act to interfere with the current dynamic or with the course the team is on.
- Intervening requires that you have observed and assessed dynamics, identified an opportunity for learning, and considered timing. It's go-time!
- You can't control your teammates. You can only invite learning and change by making the learning opportunity you have identified understood.
- There is no one right way of making an intervention. Make interventions in your own style aligned with your own universe. Be yourself!
- No need to look for the perfect intervention.
- Practice your intentional utterance.
- Assess the impact of your intervention by observing your teammates' reactions and your team's dynamic. You are back to observing dynamics and have re-entered the circular sequence of flc.
- Make a clarifying follow-up intervention if required.
- With more experience you may:
 - Consider the potential impact of your intervention on your teammates more thoroughly
 - Pay more attention on what level of system and with which intensity to intervene
 - Start using structured and conceptual interventions in addition to experiential ones¹⁰³
 - Consider how much of your own internal experience you want to share
 - Take on the facilitator role in foccs

Let me finish this chapter by stating the obvious. You will need lots of practice to become a skilled intervener. You can't practice on your own and you will require other people and their feedback. You also need to know how to be and stay fully present.

¹⁰³ see cohen-smith intervention cube in the flc toolbox chapter